

Shift Parallelism: Low-Latency, High-Throughput LLM Inference for Dynamic Workloads

Mert Hidayetoglu, Aurick Qiao, Michael Wyatt, Jeff Rasley, Yuxiong He, and Samyam Rajbhandari
Snowflake AI Research

Abstract

Efficient parallelism is necessary for achieving low-latency, high-throughput inference with large language models (LLMs). Tensor parallelism (TP) is the state-of-the-art method for reducing LLM response latency, however GPU communications reduces combined token throughput. On the other hand, data parallelism (DP) obtains a higher throughput yet is slow in response latency. Best of both worlds does not exist, and it is not possible to combine TP and DP because of the KV cache variance across the parallelisms.

We notice Sequence Parallelism (SP—Ulysses in training) has similar properties as DP but with KV cache invariance. We adapt SP to inference, and combine it with TP to get the best of both worlds. Our solution: Shift Parallelism.

Shift Parallelism dynamically switches across TP and SP, and minimizes latency in low traffic without losing throughput in high traffic. The efficient GPU communications of Shift Parallelism yields up to i) 1.51 $\times$ faster response in interactive workloads and ii) 50% higher throughput in batch workloads, compared to a TP-only solution.

We evaluate Shift Parallelism with real-world production traces with dynamic traffic patterns as well as synthetic benchmarking patterns across models, context sizes, and arrival rates. All results affirm the same: Shift Parallelism has a better latency vs. throughput tradeoff than TP or DP, and hence obtains low latency without degrading throughput in dynamic workloads.

1 Introduction

LLM inference has become the dominant workload in AI as its applications span agentic systems, chatbot (interactive) applications, model post-training (e.g. reinforcement learning), and image/video generation. The efficiency of inference systems is critical to both the performance and cost of AI applications. As of today, GPU parallelization is the prominent way of enabling large-scale AI production, and advanced multi-GPU parallelization techniques make nontrivial trade-offs across key performance metrics.

The parallelism techniques for inference are largely inherited from training, yet inference is different from training in terms of workload characteristics. Training workloads are typically homogeneous, stable, and do not care about latency but only care about throughput. Inference, on the contrary, is bursty and dynamic, and often has unpredictable traffic patterns. Furthermore, different workloads have different performance requirements. As a result, leveraging

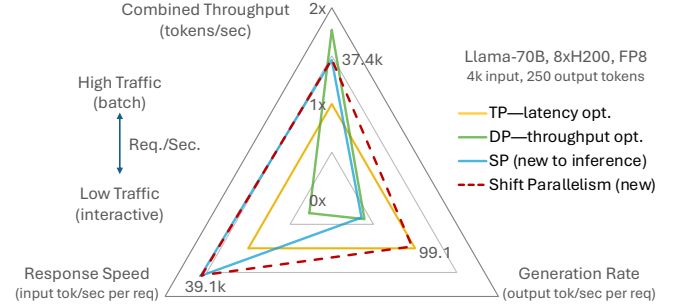


Figure 1. Comparison of response speed (#input tok./TTFT) and generation rate (1/TPOT), and throughput (tokens/sec). Shift Parallelism obtains a higher throughput than TP in high traffic, and lower latency than TP and DP in low traffic.

parallelism techniques designed for training in inference results in complex performance and cost trade-offs.

Inference Workload Characteristics

When people talk about inference systems, they often refer to interactive workloads, or batch workloads.

Interactive workloads process requests with low concurrency to minimize the completion time of each request. The completion time latency is important when there is a chain of interactions between the user and the LLM, such as in REST applications. The completion latency depends on the time to first token (TTFT)—and time per output token (TPOT), which are both critical in real-time applications.

Batch workloads involve a number of requests to be processed concurrently, and the latency of an individual request is not critical. For example, workloads such as batched summarization or translation of hundreds or thousands of documents can cause high-traffic bursts that require high combined throughput of input and output tokens to minimize the cost per token.

In enterprise systems, the request traffic pattern is often mixed and dynamically changes over time in unpredictable ways. In such dynamic settings, it is a challenge to optimize for different traffic patterns simultaneously, since existing parallelization techniques impose significant trade-offs.

Latency vs. Throughput vs. Cost Tradeoff

Existing parallelisms exhibit prohibitive latency vs. throughput (cost) trade-offs, as explained below.

Tensor parallelism (TP) partitions the model weights and computation in each layer. It has to synchronize the

轮班并行性：动态工作负载的低延迟、高吞吐量 LLM 推理

Mert Hidayetoglu、Aurick Qiao、Michael Wyatt、Jeff Rasley、Yyuxiong He 和 Samyam Rajbha
ndari Snowflake AI 研究

抽象的

高效的并行性对于通过大型语言模型 (LLM) 实现低延迟、高吞吐量推理是必要的。张量并行性 (TP) 是减少 LLM 响应延迟的最先进方法，但是 GPU 通信会降低组合令牌吞吐量。另一方面，数据并行 (DP) 获得更高的吞吐量，但响应延迟较慢。两全其美并不存在，并且由于并行性之间的 KV 缓存差异，不可能将 TP 和 DP 结合起来。

我们注意到序列并行 (SP——训练中的尤利西斯) 具有与 DP 类似的属性，但具有 KV 缓存不变性。我们使 SP 适应推理，并将其与 TP 结合起来，以获得两全其美的效果。我们的解决方案：移位并行性。

移位并行性在 TP 和 SP 之间动态切换，最大限度地减少低流量时的延迟，而不会损失高流量时的吞吐量。与仅 TP 的解决方案相比，Shift Parallelism 的高效 GPU 通信可实现 i) 交互式工作负载中的响应速度提高 1.51x，ii) 批处理工作负载中的吞吐量提高 50%。

我们利用现实世界的生产轨迹、动态流量模式以及跨模型、上下文大小和到达率的综合基准测试模式来评估移位并行性。所有结果都证实了这一点：与 TP 或 DP 相比，移位并行性具有更好的延迟与吞吐量权衡，因此可以在不降低动态工作负载吞吐量的情况下获得低延迟。

1 简介

LLM 推理已成为人工智能中的主要工作负载，因为其应用涵盖代理系统、聊天机器人（交互式）应用、模型后训练（例如强化学习）和图像/视频生成。推理系统的效率对于人工智能应用的性能和成本都至关重要。截至目前，GPU 并行化是实现大规模人工智能生产的主要方式，先进的多 GPU 并行化技术在关键性能指标之间进行了重要的权衡。

推理的并行技术很大程度上继承自训练，但推理在工作负载特征方面与训练不同。训练工作负载通常是同质的、稳定的，并且不关心延迟，只关心吞吐量。相反，推理是突发性和动态的，并且通常具有不可预测的流量模式。此外，不同的工作负载有不同的性能要求。结果，利用

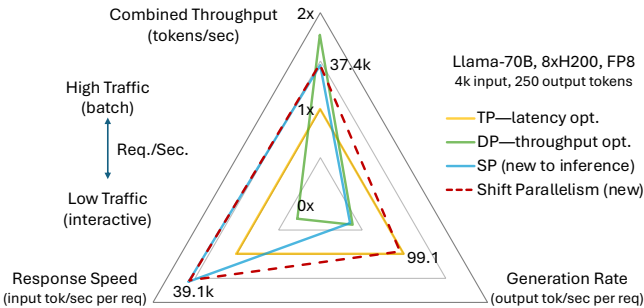


图 1. 响应速度 (#input tok./TTFT) 和生成速率 (1/TPOT) 以及吞吐量 (令牌/秒) 的比较。移位并行在高流量时比 TP 获得更高的吞吐量，在低流量时比 TP 和 DP 获得更低的延迟。

为推理训练而设计的并行技术会导致复杂的性能和成本权衡。

推理工作负载特征

当人们谈论推理系统时，他们通常指的是交互式工作负载或批处理工作负载。

交互式工作负载以低并发处理请求，以最大限度地缩短每个请求的完成时间。当用户和 LLM 之间存在一系列交互时（例如在 REST 应用程序中），完成时间延迟非常重要。完成延迟取决于第一个令牌的时间 (TTFT) 和每个输出令牌的时间 (TPOT)，这在实时应用程序中都至关重要。

批量工作负载涉及要同时处理的多个请求，并且单个请求的延迟并不重要。例如，成百上千个文档的批量摘要或翻译等工作负载可能会导致高流量突发，需要高输入和输出令牌的组合吞吐量，以最大限度地降低每个令牌的成本。

在企业系统中，请求流量模式通常是混合的，并且随着时间的推移以不可预测的方式动态变化。在这种动态设置中，同时优化不同的流量模式是一个挑战，因为现有的并行化技术需要进行重大的权衡。

延迟与吞吐量与成本的权衡

现有的并行性表现出令人望而却步的延迟与吞吐量（成本）的权衡，如下所述。

张量并行性 (TP) 划分每层中的模型权重和计算。它必须同步

Table 1. Performance tradeoffs of inference parallelisms.

Parallelism Strategy	TTFT (Latency)	Combined Throughput	TPOT (Token Latency)
Tensor Parallelism	★ Nearly Best	✗ Worst	★ Best
Data Parallelism	✗ Worst	★ Best	▼ Near Worst
Sequence Parallelism (Ulysses)	★ Best	✓ Very Good	✗ Worst
Shift Parallelism	★ Best	✓ Very Good	★ Best

embeddings across layers with costly all-reduce communications. By splitting model weights and computation across GPUs, it optimizes for latency (i.e., TTFT and TPOT), yet the communication overhead increases the cost (i.e., reduces throughput).

Data parallelism (DP) parallelizes across request boundaries in embarrassingly parallel, providing high throughput. Yet, DP cannot speed up work within a single request, and therefore unsuitable for highly interactive workloads.

The first two rows of Table 1 show the performance tradeoffs of TP and DP. We do have the choice to deploy TP and DP in separate nodes and route latency- and throughput-oriented requests, respectively. However, duplicating the node count (one for TP and one for DP) doubles the deployment cost and adds complexity.

Why can’t we combine both? A performant and low-cost inference system should be able to switch between latency-oriented and throughput-oriented parallelisms swiftly in a single deployment based on traffic demands. But this is not viable with TP and DP because they have different attention layouts. Specifically, their KV cache memory layouts are incompatible, and switching requires complex and costly data movement.

However, in this work, we notice that *Ulysses Sequence Parallelism (SP)* [7]—another form of parallelism developed and used in training—can offer a potential solution to resolving the challenge. SP splits the input sequence across GPUs to parallelize work within a single request to reduce TTFT. Unlike TP, it avoids costly all-reduce communication, while still achieving high GPU utilization. And while SP cannot parallelize decoding steps, resulting in the worst TPOT compared to TP and DP (Table 1), it has the same KV cache layout as TP, allowing for dynamic switching to TP when TPOT is critical. We call this dynamic approach Shift Parallelism, which is the focus of this work.

Shift Parallelism dynamically chooses the parallelization strategy between SP and TP based on the real-world traffic pattern, identified by the number of batched tokens in each iteration. By a given threshold, Shift Parallelism uses:

- TP for small batches—minimizing TPOT.
- SP for large batches—minimizing TTFT and achieving near-optimal throughput.

This is possible because the KV cache memory layout remains invariant between TP and SP, allowing Shift Parallelism to switch modes seamlessly, based on batch size and traffic patterns. More specifically, the KV cache layout does not change when switching across SP and TP.

Figure 1 benchmarks the latency and throughput tradeoffs of related parallelisms. Shift parallelism provides 1.5× higher throughput than TP in high traffic and 1.5× faster response in low traffic, 2× faster generation than DP in low traffic while losing only 17% throughput in high traffic.

In this paper, we

1. We characterize inference workloads and identify latency vs. throughput tradeoffs with existing inference parallelization (TP, DP).
2. Adapt SP from training to inference and generalize it for a diverse set of inference models supporting GQA [2], load-balancing at small batch sizes, combination of TP, and KV cache replication when parallelism degree is higher than number of KV heads.
3. Propose Shift Parallelism for dynamically switching across SP and TP for mitigating latency vs throughput tradeoff for dynamic workloads.
4. Test Shift Parallelism with real-world production workloads and evaluate the performance characteristics via extensive benchmarking demonstrating up to 1.5× faster response time with 50% throughput savings.
5. Open source our implementation along with other SoTA techniques that are used in practice.

The rest of the paper is organized as follows. Section 2 provides more details about production traffic patterns high-performance LLM inference. Section 3 presents SP and Shift Parallelism. Section 4 evaluates the performance of proposed techniques on real-life patterns. Section 5 about related work, and Section 6 concludes the paper.

2 Background

2.1 Production Traffic Patterns

Today, LLMs are a foundational component of AI applications. A single deployment, such as Llama-3.3-70B, can serve diverse use cases including sentiment analysis [12, 23], retrieval-augmented generation (RAG) [5, 10], coding agents [8, 22], and more. These heterogeneous use cases produce dynamic traffic patterns that must be efficiently managed by the underlying infrastructure. For example, a coding agent typically issues a small number of repeated

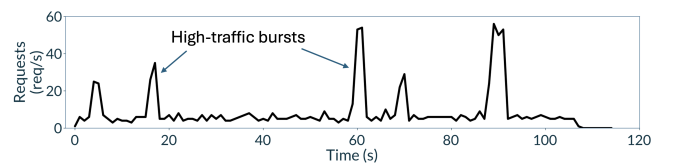

Figure 2. Bursty workload.

表 1. 推理并行性的性能权衡。

Parallelism Strategy	TTFT (Latency)	Combined Throughput	TPOT (Token Latency)
Tensor Parallelism	★ Nearly Best	✗ Worst	★ Best
Data Parallelism	✗ Worst	★ Best	▼ Near Worst
Sequence Parallelism (Ulysses)	★ Best	✓ Very Good	✗ Worst
Shift Parallelism	★ Best	✓ Very Good	★ Best

通过昂贵的全归约通信进行跨层嵌入。通过在 GPU 之间分割模型权重和计算，它优化了延迟（即 TTFT 和 TPOT），但通信开销增加了成本（即降低了吞吐量）。

数据并行性（DP）以令人尴尬的并行方式跨请求边界并行化，提供高吞吐量。然而，DP 无法加快单个请求内的工作速度，因此不适合高度交互的工作负载。

表 1 的前两行显示了 TP 和 DP 的性能权衡。我们确实可以选择在单独的节点中部署 TP 和 DP，并分别路由面向延迟和吞吐量的请求。然而，重复节点数（一个用于 TP，一个用于 DP）会使部署成本加倍并增加复杂性。

为什么我们不能将两者结合起来？高性能且低成本推理系统应该能够根据流量需求在单一部署中快速在面向延迟和面向吞吐量的并行性之间切换。但对于 TP 和 DP 来说是不可行的，因为它们有不同的注意力布局。具体来说，它们的 KV 缓存布局不兼容，并且切换需要复杂且昂贵的数据移动。

然而，在这项工作中，我们注意到尤利西斯序列并行（SP）[7]——在训练中开发和使用的另一种并行形式——可以提供解决这一挑战的潜在解决方案。SP 将输入序列拆分到多个 GPU 上，以并行化单个请求内的工作，从而减少 TTFT。与 TP 不同，它避免了成本高昂的 all-reduce 通信，同时仍然实现了高 GPU 利用率。虽然 SP 无法并行化解码步骤，导致与 TP 和 DP 相比最差的 TPOT（表 1），但它具有与 TP 相同的 KV 缓存布局，允许在 TPOT 关键时动态切换到 TP。我们将这种动态方法称为“Shift Parallelism”，这是这项工作的重点。

Shift Parallelism 根据现实世界的流量模式动态选择 SP 和 TP 之间的并行化策略，由每次迭代中批量令牌的数量来识别。通过给定阈值，移位并行度使用：

- 小批量的 TP——最小化 TPOT。
- 适用于大批量的 SP——最大限度地减少 TTFT 并实现接近最佳的吞吐量。

这是可能的，因为 KV 缓存布局在 TP 和 SP 之间保持不变，允许移位并行性根据批量大小和流量模式无缝切换模式。更具体地说，在 SP 和 TP 之间切换时，KV 缓存布局不会改变。

图 1 对相关并行性的延迟和吞吐量权衡进行了基准测试。移位并行在高流量时比 TP 提供 1.5× 高的吞吐量，在低流量时比 TP 快 1.5× 的响应速度，在低流量时比 DP 快 2× 的生成速度，而在高流量时仅损失 17% 的吞吐量。

在本文中，我们

1. 我们描述推理工作负载的特征，并通过现有推理并行化（TP、DP）确定延迟与吞吐量的权衡。
2. 将 SP 从训练到推理进行调整，并将其推广到支持 GQA [2]、小批量负载均衡、TP 组合以及并行度高于 KV 头数量时的 KV 缓存复制的多种推理模型。
3. 提出用于在 SP 和 TP 之间动态切换的移位并行性，以减少动态工作负载的延迟与吞吐量之间的权衡。
4. 使用实际生产工作负载测试轮班并行性，并通过广泛的基准测试评估性能特征，证明响应时间提高了 1.5×，吞吐量节省了 50%。
5. 开源我们的实现以及实践中使用的其他 SoTA 技术。

本文的其余部分组织如下。第 2 节提供了有关生产流量模式高性能 LLM 推理的更多详细信息。第 3 节介绍 SP 和移位并行性。第 4 节评估了所提出的技术在现实生活模式上的性能。第 5 节介绍相关工作，第 6 节总结本文。

2 背景

2.1 生产流量模式

如今，**大语言模型**为人工智能应用的基础组成部分。单个部署（例如 Llama-3.3-70B）可以服务于多种用例，包括情感分析 [12, 23]、检索增强生成 (RAG) [5, 10]、编码代理 [8, 22] 等。这些异构用例产生动态流量模式，必须由底层基础设施进行有效管理。例如，编码代理通常会发出少量重复的

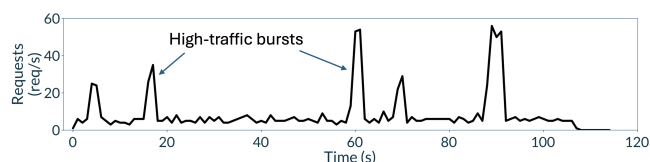


图 2. 突发工作负载。

requests in a closed loop to iteratively refine its generated code, whereas a sentiment analysis workload may submit a large batch of requests in parallel to process text stored in a database.

In production, we typically observe two main *classes* of requests. First, *interactive* or *latency-sensitive* requests (e.g., agentic or chatbot applications) generally arrive one or a few at a time, with response latencies—TTFT and TPOT (see Sec. 2.2)—directly shaping the user experience. Second, *batch* or *throughput-sensitive* requests usually arrive in large volumes (thousands to millions at once), where aggregate throughput (tokens/s) determines job completion time. When these two classes of workloads are mixed, the result is a highly *bursty* traffic pattern, with different requests subject to different quality-of-service metrics (latency versus throughput). Fig. 2 illustrates an example traffic pattern that reflects our production environment.

2.2 Performance Metrics

Since LLM use cases are diverse, metrics that measure their inference performance are also multi-faceted. In our paper, we focus on three main metrics that cover the most important aspects of interactive and batch workloads:

- **Time-to-first-token (TTFT, ms):** The time after a client submits a prompt until the first characters of response text (tokens) are received.
- **Time-per-output-token (TPOT, ms):** After the first response token is received, the time between each subsequent token until the response is completed.
- **Combined throughput (tokens/s):** The total number of tokens (both prompt and response) processed by the inference system per unit of time.

Typically, TTFT and TPOT shape the quality of service for interactive applications, while combined throughput shapes the quality of service for batch use cases and also impacts the cost of running the service for the model provider.

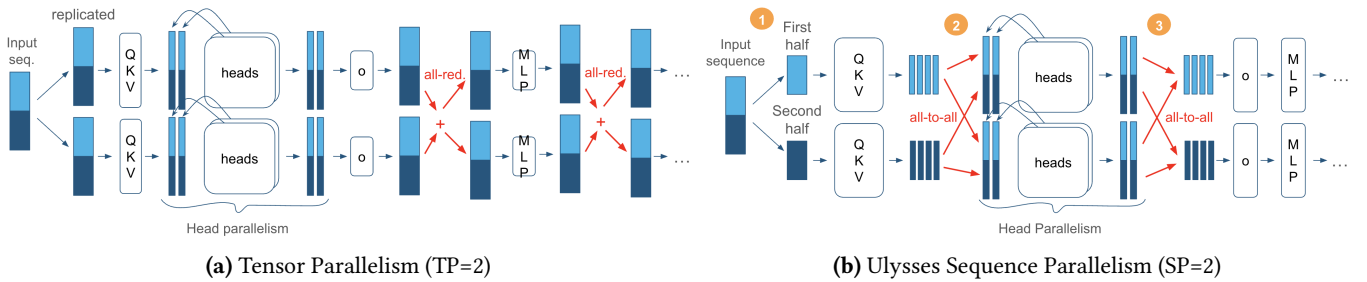


Figure 3. Parallelization of the vanilla transformer on two GPUs with TP and SP. The attention has four heads which are parallelized across heads. In (b), SP (1) partitions the input sequence, (2) switches to head parallelism using an all-to-all communication, applies head parallelization to attention, and (3) returns back to SP.

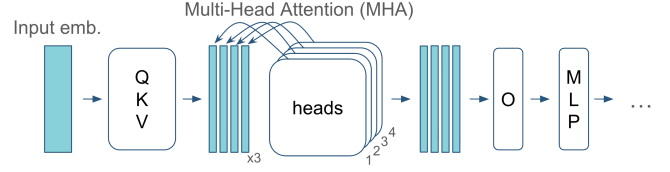


Figure 4. Vanilla transformer architecture and the attention mechanism.

2.3 Transformer Architecture

A vanilla LLM involves a series of transformer layers, and each transformer layer consists of: i) an attention mechanism and a ii) multi-layer perceptron (MLP). The weights in the transformer layer correspond to the QKV (which is a concatenation of q—query, k—key, and v—value) and O matrices in the attention, as shown in Figure 4.

First, the QKV matrix projects the input embeddings into the QKV space, where attention is applied. The Multi-Head Attention (MHA) consists multiple heads, each “attends” a different column of the input sequence. After the attention, the O matrix projects the attention output back to the embedding space to be further processed by the MLP layer.

Each LLM request involves a sequence of input and output tokens. In prefill, the input tokens are batched and propagated altogether over all of the transformer layers and initializes the KV cache for the attention layers. At the end of the prefill, the first output token is decoded according to the resulting probability distribution over all tokens in the vocabulary. For decoding the full output sequence, each new token is appended to the context sequence, and the attention patterns of subsequent context are reused from the KV cache.

2.4 Existing Parallelism Approaches

DP runs multiple replicas across requests and do not accelerate processing of a single request. For accelerating, TP partitions the weight matrices either row-wise or column-wise, as depicted in Figure 3a. Yet, row parallelization requires all-reduce with $O(n)$ communication cost, where n is the sequence length. For a fixed sequence length, the

闭环中的请求以迭代地完善其生成的代码，而情感分析工作负载可能会并行提交大量请求来处理数据库中存储的文本。

在生产中，我们通常会观察到两类主要的请求。首先，交互式或延迟敏感的请求（例如代理或聊天机器人应用程序）通常一次到达一个或几个，响应延迟——TTFT 和 TPOT（参见第 2.2 节）——直接影响用户体验。其次，批量或吞吐量敏感的请求通常会大量到达（一次数千到数百万），其中总吞吐量（令牌/秒）决定作业完成时间。当这两类工作负载混合时，结果是高度突发的流量模式，不同的请求受到不同的服务质量指标（延迟与吞吐量）的影响。图 2 展示了反映我们生产环境的示例流量模式。

2.2 性能指标

由于 LLM 用例多种多样，衡量其推理性能的指标也是多方面的。在我们的论文中，我们重点关注三个主要指标，涵盖了交互式 and 批处理工作负载最重要的方面：

- 首次令牌时间（TTFT，毫秒）：客户端提交提示后到收到响应文本（令牌）的第一个字符的时间。
- 每个输出令牌的时间（TPOT，毫秒）：收到第一个响应令牌后，每个后续令牌到响应完成之间的时间。
- 组合吞吐量（令牌/秒）：推理系统每单位时间处理的令牌（提示和响应）总数。

通常，TTFT 和 TPOT 决定交互式应用程序的服务质量，而组合吞吐量决定批量用例的服务质量，并且还会影响模型提供商运行服务的成本。

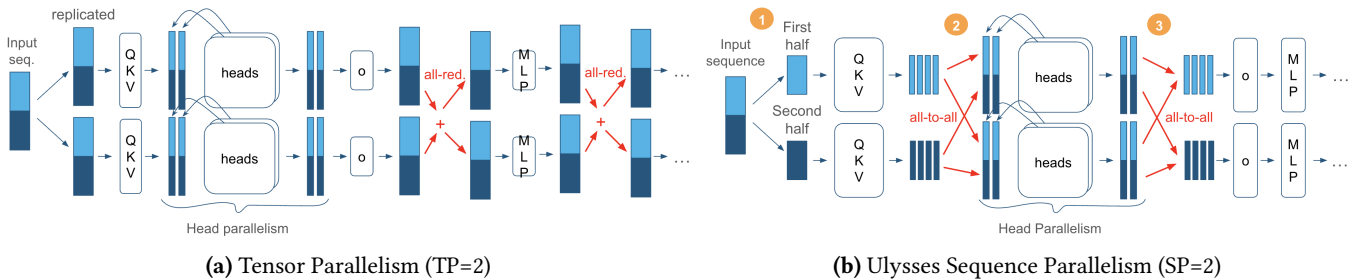


图 3. 普通 Transformer 在两个具有 TP 和 SP 的 GPU 上的并行化。注意力有四个头，它们在头之间平行。在 (b) 中，SP (1) 对输入序列进行分区，(2) 使用全对全通信切换到头并行，将头并行化应用于注意力，(3) 返回到 SP。

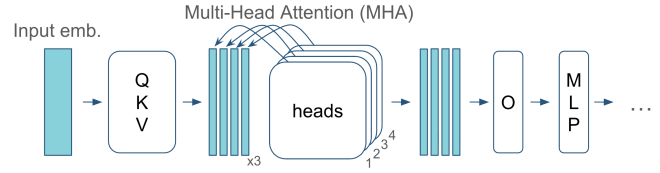


图 4. Vanilla Transformer 架构和注意力机制。

2.3 变压器架构

普通的 LLM 涉及一系列转换器层，每个转换器层由：i) 注意力机制和 ii) 多层感知器 (MLP) 组成。Transformer 层中的权重对应于注意力中的 QKV (q-query、k-key 和 v-value 的串联) 和 O 矩阵，如图 4 所示。

首先，QKV 矩阵将输入嵌入投影到 QKV 空间中，并在其中应用注意力。多头注意力 (MHA) 由多个头组成，每个头“关注”输入序列的不同列。在注意力之后，O 矩阵将注意力输出投影回嵌入空间，以由 MLP 层进一步处理。

每个 LLM 请求都涉及一系列输入和输出令牌。在预填充中，输入标记被批量处理并在所有转换器层上一起传播，并初始化注意力层的 KV 缓存。在预填充结束时，根据词汇表中所有标记的概率分布对第一个输出标记进行解码。为了解码完整的输出序列，每个新的 token 都被附加到上下文序列中，并且后续上下文的注意力模式从 KV 缓存中被重用。

2.4 现有的并行方法

DP 跨请求运行多个副本，并且不会加速单个请求的处理。为了加速，TP 按行或列对权重矩阵进行分区，如图 3a 所示。然而，行并行化需要具有 $O(n)$ 通信成本的全归约，其中 n 是序列长度。对于固定的序列长度，

Table 2. Computational Complexity of TP and SP.

	Per-GPU Complexity			
	Memory	Compute	Comm. Volume	Comm./Compute
TP	$m(n,w)/TP$	$f(n,w)/TP$	$c(n,w)$	$TP \times \text{const}$
SP	$m(n,w)$	$f(n,w)/SP$	$c(n,w)/SP$	const

n : sequence length, w : # parameters

communication-to-compute ratio increases with the TP degree as shown in the last column of Table 2.

Head Parallelism is commonly used with SP and TP, where the attention heads are distributed across the GPUs equally. This is done with no additional cost by column-wise partitioning of the QKV matrix, as Figure 3a shows. Head parallelism cannot be scaled beyond the number of heads.

Ulysses Sequence Parallelism partitions the embedding sequence for parallelizing inference. Yet, each attention head requires the full sequence, resulting in all-to-all communications before and after the attention layer, as shown in Figure 3b. Nevertheless, the communication cost does not increase with SP as shown in the last column of Table 2.

Ulysses has been applied to training, and in this work, we extend Ulysses for inference handling inference-specific nuances to allow for a generalized implementation¹.

3 System Design and Implementation

3.1 Overview

We design Shift Parallelism to enable switching between SP and TP, addressing the latency-throughput tradeoff in inference. The key insight is that both configurations must share the same KV cache layout—what we call KV cache invariance. This invariance allows us to switch seamlessly between SP and TP.

Figure 5 illustrates KV cache invariance between $TP=2$ and $SP=2$. In the center, four attention heads are evenly distributed across two GPUs (two heads per GPU). This distribution is identical under both $TP = 2$ and $SP = 2$, allowing the two configurations to share a single attention mechanism and KV cache.

3.1.1 SP for Inference. Applying SP to inference is more nuanced than in training because of variable traffic patterns (e.g., load imbalance) and the lack of Grouped Query Attention (GQA) support in earlier designs, and parallelism that can exceed the number of KV attention heads. To address this, we develop a fully generic SP for inference that: i) supports GQA, ii) replicates KV cache as needed, iii) handles load balancing under low-traffic scenarios.

Furthermore, real-world inference is not simply a choice between SP or TP. For optimal performance, systems require arbitrary combinations of SP and TP. Our design supports this flexibility, enabling mixed (SP, TP) configurations.

¹In the rest of the paper, we use Ulysses and SP interchangeably.

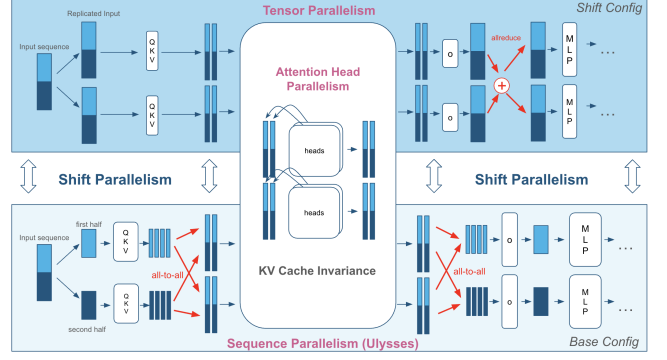


Figure 5. Although SP and TP are essentially different parallelisms, Shift Parallelism exploits the KV cache invariance between SP and TP for swiftly switching between them.

3.1.2 Shift Parallelism. Building on this flexible SP design, we implement Shift Parallelism using two configurations:

1. *Base configuration:* Uses either full SP or a mixed (SP, TP) setup, as long as $SP \times TP = P$, where P is the total number of GPUs in the node.
2. *Shift configuration:* Always ($SP = 1, TP = P$), spanning the full node.

While conceptually simple, several technical challenges arise in enabling efficient transitions between these configurations:

Cache invariance: In general, the base and shift configurations are not automatically invariant. For arbitrary (SP, TP) combinations, head ordering in head parallelism breaks the invariance. We resolve this by developing a general process-to-data mapping to ensure KV cache consistency in Section 3.3.1.

Weight handling: Transitioning between configurations requires that weights be compatible across both base and shift modes. We consider two strategies: (i) on-the-fly slicing, and (ii) explicit weight replication. Based on memory cost analysis, we adopt the latter as the preferred approach discussed in Section 3.3.2.

We designed Shift Parallelism for ease of use and adoption. It is integrated into vLLM via a plug-in system (Section 3.4) and is already deployed in production.

The rest of this section dives into the design details of SP and Shift Parallelism for inference.

3.2 SP for Inference

SP is essential for Shift Parallelism because it is the throughput-optimized counterpart of TP as they have the same KV cache layout, and therefore we can switch between them without changing the attention mechanism.

3.2.1 Design for General Inference. SP is originally implemented for training, and lacks important components,

表 2. TP 和 SP 的计算复杂度。

	Per-GPU Complexity			
	Memory	Compute	Comm. Volume	Comm./Compute
TP	$m(n,w)/TP$	$f(n,w)/TP$	$c(n,w)$	$TP \times \text{const}$
SP	$m(n,w)$	$f(n,w)/SP$	$c(n,w)/SP$	const

n: 序列长度, w: #参数

通信与计算之比随着 TP 程度的增加而增加, 如表 2 最后一列所示。

头并行通常用于 SP 和 TP, 其中注意力头均匀分布在 GPU 上。这是通过 QKV 矩阵的列分区来完成的, 无需额外成本, 如图 3a 所示。磁头并行度不能超出磁头数量。

Ulysses Sequence Parallelism 划分嵌入序列以实现并行推理。然而, 每个注意力头都需要完整的序列, 从而在注意力层之前和之后产生全面的通信, 如图 3b 所示。尽管如此, 通信成本并没有随着 SP 的增加而增加, 如表 2 最后一列所示。

Ulysses 已应用于训练, 在这项工作中, 我们将 Ulysses 扩展为推理处理特定推理的细微差别, 以允许通用实现¹。

3 系统设计与实现

3.1 概述

我们设计了 Shift Parallelism 来实现 SP 和 TP 之间的切换, 解决推理中的延迟-吞吐量权衡问题。关键的一点是, 两种配置必须共享相同的 KV 缓存布局, 即我们所说的 KV 缓存不变性。这种不变性使我们能够在 SP 和 TP 之间无缝切换。

图 5 说明了 TP=2 和 SP=2 之间的 KV 缓存不变性。在中心, 四个注意力头均匀分布在两个 GPU 上 (每个 GPU 两个注意力头)。这种分布在 TP=2 和 SP=2 下是相同的, 允许这两种配置共享单个注意力机制和 KV 缓存。

3.1.1 用于推理的 SP。将 SP 应用于推理比在训练中更加细致, 因为早期设计中存在可变的流量模式 (例如负载不平衡) 和缺乏分组查询注意力 (GQA) 支持, 并且并行性可能超过 KV 注意力头的数量。为了解决这个问题, 我们开发了一个完全通用的 SP 来进行推理: i) 支持 GQA, ii) 根据需要复制 KV 缓存, iii) 处理低流量场景下的负载平衡。

此外, 现实世界的推理不仅仅是 SP 或 TP 之间的选择。为了获得最佳性能, 系统需要 SP 和 TP 的任意组合。我们的设计支持这种灵活性, 支持混合 (SP, TP) 配置。

¹In the rest of the paper, we use Ulysses and SP interchangeably.

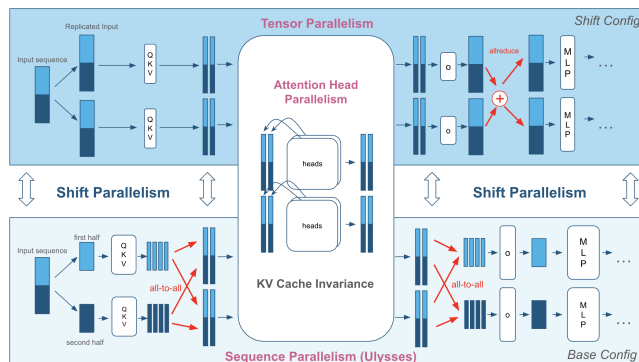


图 5. 虽然 SP 和 TP 本质上是不同的并行性, 但移位并行性利用 SP 和 TP 之间的 KV 缓存不变性来在它们之间快速切换。

3.1.2 移位并行性。基于这种灵活的 SP 设计, 我们使用两种配置实现移位并行性:

1. 基本配置: 使用完整 SP 或混合 (SP, TP) 设置, 只要 $SP \times TP = P$, 其中 P 是节点中 GPU 的总数。
2. 班次配置: 始终为 ($SP = 1, TP = P$), 跨越整个节点。

虽然概念上很简单, 但在实现这些配置之间的高效转换时会出现一些技术挑战:

缓存不变性: 一般来说, 基本配置和移位配置不会自动保持不变。对于任意 (SP, TP) 组合, 头并行性中的头排序会破坏不变性。我们通过开发通用的进程到数据映射来解决这个问题, 以确保第 3.3.1 节中的 KV 缓存一致性。

重量处理: 配置之间的转换要求重量在基本模式和存档模式之间兼容。我们考虑两种策略: (i) 即时切片, (ii) 显式权重复制。基于内存成本分析, 我们采用后者作为第 3.3.2 节中讨论的首选方法。

我们设计 Shift Parallelism 是为了易于使用和采用。它通过插件系统 (第 3.4 节) 集成到 vLLM 中, 并已部署在生产中。

本节的其余部分将深入探讨 SP 和 Shift Parallelism 的设计细节以进行推理。

3.2 推理 SP

SP 对于移位并行性至关重要, 因为它是 TP 的吞吐量优化对应物, 因为它们具有相同的 KV 缓存布局, 因此我们可以在它们之间切换而不改变注意力机制。

3.2.1 一般推理设计。SP 最初是为了训练而实现的, 缺乏重要的组成部分,

such as GQA mechanism [2], that is commonly used in inference models. The original MHA mechanism is described in Section 2.4.

GQA Extension: In this work, we extend SP for GQA mechanism for adapting SP to a diverse set of LLMs that are used in inference. GQA saves memory by sharing each KV head with multiple query heads. However, multi-GPU scaling of GQA is nontrivial with models involving small #KV heads. For example, Qwen-30B-A3B attention has 4 KV heads, and it cannot be scaled to an 8×GPU node since there are not enough KV heads to be distributed across more than 4 GPUs.

KV Cache Replication: TP solves the scaling problem by replicating the KV weights in the QKV projection (see Section 2.3), which recomputes the KV cache within GQA groups redundantly. This solution is not applicable to SP because each process owns only a slice of the input sequence and the missing slices cannot be replicated simply by recomputation.

For SP in inference, we implemented the KV cache replication using all-to-all communications. The KV heads are replicated within the send buffers of the collective call, resulting in replication in the receiving buffers across GPUs.

Fusing Communications: The QKV matrix fuses operations related to q, k, and v, bringing multiple communications down to a single matrix all-to-all communications together as represented in Algorithm 1 Line 4.

The GQA implementation replaces $3 \times h$ with $h + 2 \times h_{kv}$, where h is the # q heads and h_{kv} is the # k and v heads, and replicates the KV cache if necessary for handling any combination of h and h_{kv} with a single, e.g., fused, all-to-all communications.

Small Batch Size and Load Imbalance: The main problem with SP is the load imbalance with small batch sizes, i.e., n is small in Algorithm 1. This problem does not exist in training because batches are large and static, whereas in inference, the batch size varies according to the traffic.

Specifically, decoding in low traffic yields small batch sizes because there are only a few tokens produced at a time. Small batch sizes comparable to the SP degree cannot be evenly partitioned across GPUs, causing serious load imbalance. For example, when the batch size is 9 and $SP = 8$, all GPUs will process a single token except the one that processes two tokens, causing 50% efficiency. SP even breaks down when $SP > \text{batch size}$, causing sparse communications.

To provide load balancing, we pad batches up to a multiple of SP degree so that we can evenly distribute them. Nevertheless, the padding results in redundant tokens, yielding a longer TPOT and hence longer request completion time compared to TP in low-traffic decoding.

3.2.2 Combined (SP, TP) Algorithm. We need to combine SP with TP for handling large models that do not fit (or barely fits) in a single GPU. For a throughput-optimal config,

Algorithm 1 Combined (SP, TP) for the base config.

```

1:  $embed[n/SP, d] \leftarrow SP.slice(input\_embeds[n, d])$ 
2: for  $i = 1, \dots, L$  do
3:    $qkv\_heads[n/SP, 3 \times h/TP] \leftarrow embed * layer_i.qkv[d, 3 \times h/TP]$ 
4:    $qkv\_heads[n, 3 \times h/(SP \times TP)] \leftarrow SP.all\_to\_all(qkv\_heads)$ 
5:    $attn\_o[n, h/(SP \times TP)] \leftarrow layer_i.attn(qkv\_heads)$ 
6:    $attn\_o[n/SP, h/TP] \leftarrow SP.all\_to\_all(attn\_o)$ 
7:    $embed[n/SP, d] \leftarrow attn\_o * layer_i.o[h/TP, d]$ 
8:    $TP.all\_reduce(embed)$ 
9:    $act[n/SP, d'/TP] \leftarrow embed * layer_i.mlp\_up[d, d'/TP]$ 
10:   $embed[n/SP, d] \leftarrow act * layer_i.mlp\_down[d'/TP, d]$ 
11:   $TP.all\_reduce(embed)$ 
12: end for
13:  $output\_embeds[n, d] \leftarrow SP.all\_gather(embed[n/SP, d])$ 
14: return  $output\_embeds$ 

```

we avoid partitioning the model with TP as much as each partition fits into GPU memory, and there is enough room for KV cache for providing concurrency and high throughput. Then the rest of the GPUs can be efficiently employed using SP, which enlarges KV cache. For example, our evaluation involves Llama-17B-16E (FP8) has 109 GB memory footprint, yet needs at least $TP = 2$ for processing long contexts concurrently within 141 GB GPU memory, and therefore we need a combination of ($TP = 2, SP = 4$) for an optimal deployment on a node with 8 GPUs.

Algorithm 1 shows the forward pass algorithm with an arbitrary (SP, TP) configuration, where n is the sequence length, d is the hidden dimension, and h is the number of heads.

3.3 Shift Parallelism

Shift parallelism (the main contribution of this paper) is designed to obtain low latency (TTFT and TPOT) in low traffic, and high throughput in high traffic by switching across parallelisms. The optimal parallelisms that we cover with Shift Parallelism are summarized in Table 3.

We can apply shift parallelism only across TP and SP because of their KV cache invariance property (Section 3.3.1), as a result, Shift Parallelism provides superior performance in the highlighted cases. The only case Shift Parallelism loses on DP (but wins on TP) is the throughput in high traffic, because parallel attention inevitably requires GPU communications.

In shift parallelism, we have two configurations; a) the base config that implements SP to optimize TTFT and throughput, and b) the shift configuration that implements full TP to optimize TPOT. The base configuration can optionally be

Table 3. Optimal Parallelisms Covered by Shift Parallelism.

	Low Traffic	High Traffic
TTFT	SP	SP
TPOT	TP	SP
Throughput	SP* or TP	DP

*SP for long input, TP for long output.

例如推理模型中常用的GQA机制[2]。最初的MHA机制在2.4节中描述。

GQA 扩展：在这项工作中，我们扩展了GQA机制的SP，以使SP适应推理中使用的各种LLM集。GQA通过与多个查询头共享每个KV头来节省内存。然而，对于涉及小#KV头的模型来说，GQA的多GPU扩展并非易事。例如，Qwen-30B-A3B注意力有4个KV头，它无法扩展到8×GPU节点，因为没有足够的KV头来分布在超过4个GPU上。

KV 缓存复制：TP通过复制QKV投影中的KV权重（参见第2.3节）来解决缩放问题，这会冗余地重新计算GQA组内的KV缓存。该解决方案不适用于SP，因为每个进程仅拥有输入序列的一个切片，并且不能简单地通过重新计算来复制丢失的切片。

对于推理中的SP，我们使用全对全通信实现了KV缓存复制。KV头在集体调用的发送缓冲区中进行复制，从而导致在GPU上的接收缓冲区中进行复制。

融合通信：QKV矩阵融合了与q、k和v相关的操作，将多个通信简化为单个矩阵的所有通信，如算法1第4行所示。

GQA实现将 $3 \times h$ 替换为 $h + 2 \times h_{kv}$ ，其中 h 是#q头， h_{kv} 是#k和v头，并在必要时复制KV缓存，以通过单个（例如，融合的、全对所有通信）处理 h 和 h_{kv} 的任意组合。

小批量和负载不平衡：SP的主要问题是小幅量的负载不平衡，即算法1中的 n 很小。这个问题在训练中不存在，因为批量很大并且是静态的，而在推理中，批量大小根据流量而变化。

具体来说，低流量下的解码会产生较小的批量，因为一次仅产生几个令牌。与SP程度相当的小批量大小无法在GPU之间均匀划分，从而导致严重的负载不平衡。例如，当批量大小为9和 $SP = 8$ 时，除了处理两个令牌的GPU之外，所有GPU都将处理单个令牌，导致效率为50%。SP甚至在 $SP >$ 批量大小时崩溃，导致通信稀疏。

为了提供负载平衡，我们将批次填充到SP程度的倍数，以便我们可以均匀地分配它们。然而，填充会导致冗余令牌，从而产生更长的TPOT，因此与低流量解码中的TP相比，请求完成时间更长。

3.2.2 组合 (SP, TP) 算法。我们需要将SP与TP结合起来，以处理不适合（或几乎不适合）单个GPU的大型模型。对于吞吐量最佳配置，

Algorithm 1 Combined (SP, TP) for the base config.

```

1:  $embed[n/SP, d] \leftarrow SP.slice(input\_embeds[n, d])$ 
2: for  $i = 1, \dots, L$  do
3:    $qkv\_heads[n/SP, 3 \times h/TP] \leftarrow embed * layer_i.qkv[d, 3 \times h/TP]$ 
4:    $qkv\_heads[n, 3 \times h/(SP \times TP)] \leftarrow SP.all\_to\_all(qkv\_heads)$ 
5:    $attn\_o[n, h/(SP \times TP)] \leftarrow layer_i.attn(qkv\_heads)$ 
6:    $attn\_o[n/SP, h/TP] \leftarrow SP.all\_to\_all(attn\_o)$ 
7:    $embed[n/SP, d] \leftarrow attn\_o * layer_i.o[h/TP, d]$ 
8:    $TP.all\_reduce(embed)$ 
9:    $act[n/SP, d'/TP] \leftarrow embed * layer_i.mlp\_up[d, d'/TP]$ 
10:   $embed[n/SP, d] \leftarrow act * layer_i.mlp\_down[d'/TP, d]$ 
11:   $TP.all\_reduce(embed)$ 
12: end for
13:  $output\_embeds[n, d] \leftarrow SP.all\_gather(embed[n/SP, d])$ 
14: return  $output\_embeds$ 

```

我们避免使用TP对模型进行分区，只要每个分区适合GPU内存，并且有足够的空间用于KV缓存以提供并发性和高吞吐量。然后可以使用SP来有效地利用其余的GPU，从而扩大KV缓存。例如，我们的评估涉及Llama-17B-16E (FP8)，其内存占用为109 GB，但至少需要 $TP = 2$ 才能在141 GB GPU内存内同时处理上下文，因此我们需要($TP = 2, SP = 4$)的组合才能在具有8个GPU的节点上实现最佳部署。

算法1显示了具有任意(SP, TP)配置的前向传递算法，其中 n 是序列长度， d 是隐藏维度， h 是头数。

3.3 移位并行性

移位并行（本文的主要贡献）旨在通过并行切换来在低流量下获得低延迟（TTFT和TPOT），并在高流量下获得高吞吐量。表3总结了我们使用Shift Parallelism涵盖的最佳并行性。

由于它们的KV缓存不变性（第3.3.1节），我们只能在TP和SP之间应用移位并行性，因此，移位并行性在突出显示的情况下提供了卓越的性能。Shift Parallelism在DP上失败（但在TP上获胜）的唯一情况是高流量时的吞吐量，因为并行注意力不可避免地需要GPU通信。

在移位并行性中，我们有两种配置；a) 实现SP以优化TTFT和吞吐量的基本配置，以及b) 实现完整TP以优化TPOT的移位配置。基本配置可以选择

表3. 移位并行性涵盖的最佳并行性。

	Low Traffic	High Traffic
TTFT	SP	SP
TPOT	TP	SP
Throughput	SP* or TP	DP

*SP表示长输入，TP表示长输出。

a combination of TP and SP (Section 3.2.2), if the model does not fit into a single GPU.

How do we shift? The main criterion of switching between configurations is simple: We choose the base model for large batch size and the shift model for small batch size. Therefore, we decide on a shift parallelism threshold, if the batch size is larger than the threshold, we choose (SP, TP) configuration, and choose the shift configuration, i.e., full-TP on $(SP \times TP)$ group as Algorithm 2 describes.

Algorithm 2 Shift parallel ($SP \times TP$) forward pass.

```

1: if  $n > \text{threshold}$  then
2:   return Algorithm 1[ $SP, TP$ ](input_embed[n, d])
3: else
4:   return Algorithm 1[1,  $SP \times TP$ ](input_embed[n, d])
5: end if

```

3.3.1 General KV Cache Invariance. The KV cache invariance does not only require the same attention head layouts, but also the same ordering of the heads. Interestingly, the invariance across SP and TP breaks down when shifting across arbitrary (SP, TP) and $(SP \times TP)$ configurations. When the base configuration involves a combination of SP and TP, e.g., $(SP = 3, TP = 2)$ the attention head order does not follow the same order as in $TP = 6$, i.e., not $(0, 1, 2, 3, 4, 5)$, anymore but $(0, 2, 4, 1, 3, 5)$. The attention mechanism does not care about the order of the heads as long as we lock in to an order (i.e., base config’s) and stick to it when switching to the shift config as depicted in Figure 6.

Figure 6 shows the distributed memory layout of Q projection with the (a) base config and (b) the shift config. The original config yields TP groups $(0, 1)$, $(2, 3)$, $(4, 5)$ and SP groups $(0, 2, 4)$, $(1, 3, 5)$. TP groups partition the Q weights across heads (i.e. columns) and replicate the input embeddings. SP groups partition the embeddings across the sequence (i.e., rows), and replicate the Q weights. As a result of the 2D partitioning, the output of the linear layer ($q_{\perp}$) has a global layout as shown in the figure. As a result of the all-to-all communication within the SP groups, the head partitions (q) have interleaved head ordering $(0, 2, 4, 1, 3, 5)$. We need to adjust the attention head ordering of the shift config accordingly to provide KV cache consistency.

3.3.2 Memory Management. There are two ways of implementing Shift Parallelism with generalized KV cache invariance. 1) slicing the model weights on-the-fly and 2) loading separate models that share the attention mechanism (and the KV cache). We use 2) in our implementation.

On-the-fly slicing. This implementation modifies the linear layer implementation of the original code such that each GPU multiplies a slice of the base model’s weight partition. To preserve KV cache invariance, each GPU must have the slice according to their SP ranks. For example, global ranks

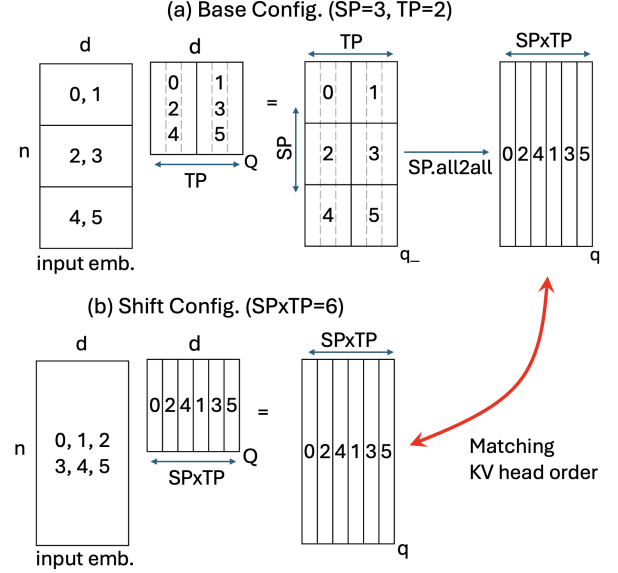


Figure 6. KV cache invariance of six heads across the base and shift configs. The shift config should shard the Q weights according to SP and TP degrees of the base config.

2, and 3 in Figure 6 gets heads 1, and 4 which are already in the base model’s respective weight partition.

Slicing provides the same effect with TP, and has no memory overhead since the running buffer can be reused for all layers. Nevertheless, it is not as performant as the next solution because each slicing requires matrix transposition due to an FP8 hardware limitation of Hopper tensor cores.

Separate Models. The separate model solution does not share the weights across the base and shift configs, but replicates the weights. In this implementation, we load two separate models, one for the base configuration and one for the shift configuration, and these models share the same KV cache.

When loading the weights for the shift model, we use a separate group, SP_TP that spans both SP and TP groups, but with the order of SP group to preserve KV cache coherency. As a result, the shift model will load the right weight shards as shown in Figure 6.

- TP: $[[0, 1], [2, 3], [4, 5]]$
- SP: $[[0, 2, 4], [1, 3, 5]]$
- SP_TP : $[[0, 2, 4, 1, 3, 5]]$

With the separate model solution, we can write the total weight footprint as

$$w_{total} = \frac{w_{base}}{TP} + \frac{w_{base}}{SP \times TP}, \quad (1)$$

where the first and second terms represent the base and shift models’ weights, respectively. As a results, the memory overhead of the shift model is $1/SP$, i.e., a base model with more SP and less TP alleviates the memory overhead of the

如果模型不适合单个 GPU，则使用 TP 和 SP 的组合（第 3.2.2 节）。

我们如何转变？配置之间切换的主要标准很简单：我们选择大批量的基本模型和小批量的转移模型。因此，我们决定一个移位并行度阈值，如果批量大小大于阈值，我们选择 (SP, TP) 配置，并选择移位配置，即 $(SP \times TP)$ 组上的全 TP，如算法 2 所述。

Algorithm 2 Shift parallel $(SP \times TP)$ forward pass.

```

1: if  $n > \text{threshold}$  then
2:   return Algorithm 1[ $SP, TP$ ]( $\text{input\_embed}[n, d]$ )
3: else
4:   return Algorithm 1[1,  $SP \times TP$ ]( $\text{input\_embed}[n, d]$ )
5: end if

```

3.3.1 一般 KV 缓存不变性。KV 缓存不变性不仅需要相同的注意力头布局，而且还需要相同的头排序。有趣的是，当在任意 (SP, TP) 和 $(SP \times TP)$ 配置之间转换时，SP 和 TP 之间的不变性就会崩溃。当基本配置涉及 SP 和 TP 的组合时，例如 $(SP = 3, TP = 2)$ ，注意力头顺序不再遵循与 $TP = 6$ 中相同的顺序，即不再是 $(0, 1, 2, 3, 4, 5)$ ，而是 $(0, 2, 4, 1, 3, 5)$ 。注意力机制并不关心头部的顺序，只要我们锁定一个顺序（即基本配置）并在切换到班次配置时坚持它，如图 6 所示。

图 6 显示了具有 (a) 基本配置和 (b) 移位配置的 Q 投影的分布式内存布局。原始配置生成 TP 组 $(0, 1), (2, 3), (4, 5)$ 和 SP 组 $(0, 2, 4), (1, 3, 5)$ 。TP 组将 Q 权重划分为头（即列）并复制输入嵌入。SP 组在序列（即行）上划分嵌入，并复制 Q 权重。由于二维分区，线性层（ $q_{\perp}$ ）的输出具有如图所示的全局布局。由于 SP 组内的全对全通信，头分区（ q ）具有交错的头排序 $(0, 2, 4, 1, 3, 5)$ 。我们需要相应地调整移位配置的注意力头排序以提供 KV 缓存一致性。

3.3.2 内存管理。有两种方法可以实现具有广义 KV 缓存不变性的移位并行性。1) 动态切片模型权重，2) 加载共享注意力机制（和 KV 缓存）的单独模型。我们在实现中使用 2)。

即时切片。此实现修改了原始代码的线性层实现，以便每个 GPU 乘以基本模型权重分区的一个切片。为了保持 KV 缓存不变性，每个 GPU 必须根据其 SP 等级拥有切片。例如，全球排名

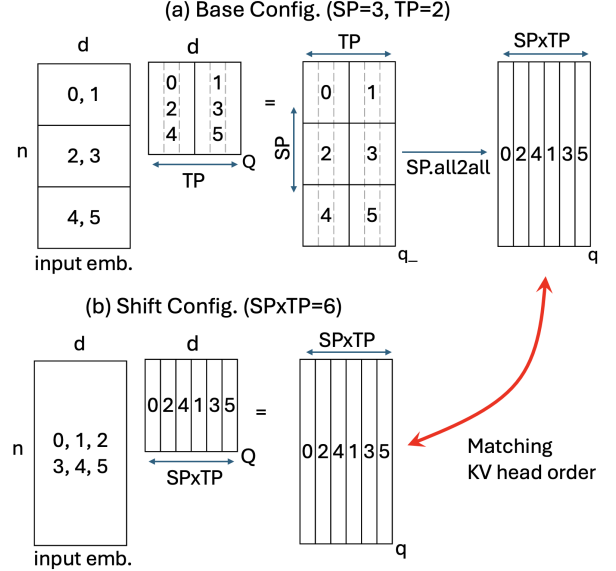


图 6. 基本配置和移位配置中六个磁头的 KV 缓存不变性。移位配置应根据基本配置的 SP 和 TP 度数对 Q 权重进行分片。

图 6 中的 2 和 3 获得头 1 和 4，它们已经位于基本模型各自的权重分区中。

切片提供了与 TP 相同的效果，并且没有内存开销，因为运行缓冲区可以为所有层重用。然而，它的性能不如下一个解决方案，因为由于 Hopper 张量核心的 FP8 硬件限制，每个切片都需要矩阵转置。

单独的模型。单独的模型解决方案不会在基本配置和移位配置之间共享权重，而是复制权重。在此实现中，我们加载两个单独的模型，一个用于基本配置，一个用于移位配置，并且这些模型共享相同的 KV 缓存。

当加载移位模型的权重时，我们使用一个单独的组 SP_TP ，它跨越 SP 组和 TP 组，但使用 SP 组的顺序来保持 KV 缓存一致性。因此，班次模型将加载正确的权重分片，如图 6 所示。

- TP: $[[0, 1], [2, 3], [4, 5]]$
- SP: $[[0, 2, 4], [1, 3, 5]]$
- SP_TP : $[[0, 2, 4, 1, 3, 5]]$

通过单独的模型解决方案，我们可以将总重量足迹写为

$$w_{total} = \frac{w_{base}}{TP} + \frac{w_{base}}{SP \times TP}, \quad (1)$$

其中第一项和第二项分别表示基础模型和移位模型的权重。结果，移位模型的内存开销为 $1/SP$ ，即具有更多 SP 和更少 TP 的基础模型减轻了移位模型的内存开销

shift model. For example, when $SP = 8$, the shift model’s memory overhead is 12.5%.

3.4 Integration into vLLM

None of the existing inference frameworks implement SP (Ulysses), and also modifying existing enterprise frameworks (such as vLLM) is tedious. We overcame the problem by developing a plug-in system [20] for implementing the proposed techniques in this paper.

For achieving low latency in inference, it is crucial to enable compilation and CUDA graph capture mechanisms in vLLM. The plug-in system compiles and captures both base model and shift model separately. Capturing separate graphs for multiple shapes yields hundreds of graphs, which are registered during initialization and replayed accordingly at runtime. The additional graphs for the shift model do not increase the capturing time or memory significantly.

4 Evaluation

In this section, we demonstrate that Shift-Parallelism can mitigate the latency vs throughput tradeoffs commonly seen in TP and DP. More specifically, we show

1. Shift Parallelism can adapt to bursty synthetic traffic pattern, achieving simultaneously lowest latency (up to $3.23\times$ lower) and near optimal throughput compared to TP and DP.
2. On open-source production traces, Shift Parallelism:
 - Consistently obtains the lowest TTFT and TPOT, and hence completion time statistics compared to TP and DP when running Azure LLM Code Trace [15].
 - Can keep up with the request traffic with no wait time whereas TP and DP cause growing wait times when running Mooncake Conversation Trace [17].
3. Extensive evaluation of Shift Parallelism over a wide range of sequences and request traffic demonstrating consistently superior performance ($1.67\times$ – $6.97\times$ faster response, $1\times$ – $2.45\times$ faster generation and $1.51\times$ higher throughput) compared to TP and DP, guaranteeing lowest latency with low cost over the entire spectrum, even in high traffic.
4. Shift Parallelism can accelerate real-world production deployment offering fastest open-source inference solution ($3.4\times$ lower completion time, and $1.06\times$ higher throughput) by composing with SoTA inference technologies like SwiftKV and Speculative Decoding.
5. The cost breakdown analysis of DP, TP, and Shift Parallelism and explore further tradeoffs across dense and sparse models.

4.1 Experimental Setup

4.1.1 Hardware. Unless specified, we use AWS instances with 8xH200 GPUs each, i.e., p5en.48xlarge. Each GPU has 141 GB memory with 4.8 TB/s bandwidth, and also provides

a peak dense matrix multiplication of 1,979 FP8 TFLOPS with tensor cores. The GPUs are interconnected with an NVSwitch network with 900 GB/s rated bandwidth.

4.1.2 Software. Unless specified, we use our implementation (Sec. 3.4) plugged into vLLM v0.9.2. For comparison, we use SGLang [24] v0.4.6, TRT-LLM [13] v0.18.2.

Table 4. Models used in evaluation.

Model Name	Num. Params.	Num. Lay.	Hidden Size	# Heads Q	# Heads KV
Llama-70B	70B	80	8192	64	8
Qwen-32B	32B	64	5120	64	8
Llama-17B-16E	109B/17B	48	5120	40	8
Qwen-30B-A3B	30B/3B	48	2048	32	4

4.1.3 Models. We use the models listed in Table 4, all with FP8 quantization. Shift Parallelism is originally designed for dense models, therefore we first present the main evaluation for L70B and Q32B, and then we discuss the performance limitations with mixture of experts (MoE) models—Q30B-A3B and L17B-16E—their static and active number of parameters are shown separately in Table 4.

4.1.4 Datasets. We use the following datasets in the order of presentation. i) A bursty synthetic pattern that resembles real-life production environment, ii) a couple of open-source production traces (details are given in Section 4.2.2),

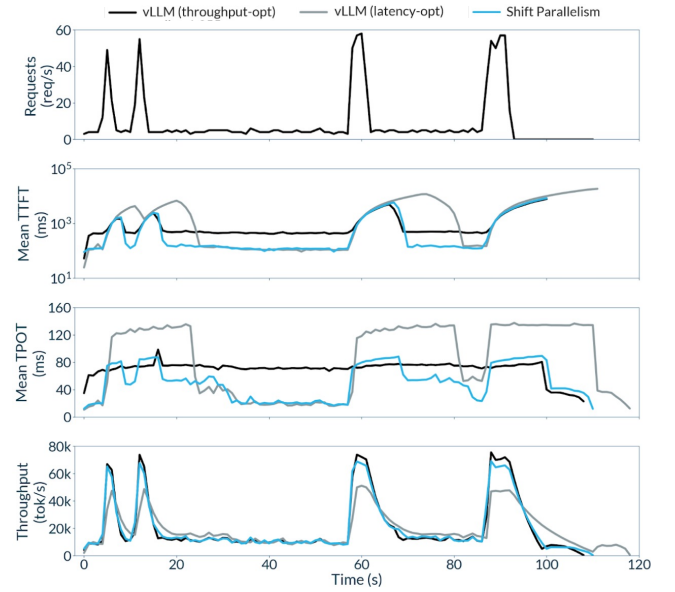


Figure 7. Shift Parallelism achieves the lowest response, fastest generation and near-optimal throughput under dynamic traffic. We used Llama-70B and a modified version vLLM’s serving benchmark that makes requests a steady stream of request at low frequency with occasional bursts of high frequency requests.

转变模型。例如，当 $SP = 8$ 时，移位模型的内存开销为12.5%。

3.4 融入vLLM

现有的推理框架都没有实现SP (Ulysses)，而且修改现有的企业框架（例如vLLM）也很繁琐。我们通过开发一个插件系统[20]来实现本文提出的技术，从而克服了这个问题。

为了实现低延迟推理，在 vLLM 中启用编译和 CUDA 图捕获机制至关重要。插件系统分别编译和捕获基本模型和转换模型。捕获多个形状的单张图会产生数百个图形，这些图形在初始化期间注册并在运行时相应地重播。班次模型的附加图表不会显着增加捕获时间或内存。

4 评价

在本节中，我们将证明移位并行可以减轻 TP 和 DP 中常见的延迟与吞吐量的权衡。更具体地说，我们展示

1. 与 TP 和 DP 相比，移位并行可以适应突发合成流量模式，同时实现最低延迟（低至 3.23x）和接近最佳吞吐量。

2. 在开源生产痕迹上，Shift Parallelism:

- 在运行 Azure LLM 代码跟踪 [15] 时，与 TP 和 DP 相比，始终获得最低的 TTFT 和 TPOT，以及完成时间统计信息。
- 可以跟上请求流量，无需等待时间，而 TP 和 DP 在运行 Mooncake Conversation Trace [17] 时会导致等待时间增加。

3. 与 TP 和 DP 相比，对各种序列和请求流量上的移位并行性进行了广泛的评估，显示出始终如一的卓越性能（响应速度加快 1.67x–6.97x，生成速度加快 1x–2.45x，吞吐量提高 1.51x），即使在高流量下，也能保证在整个频谱上以低成本实现最低延迟。

4. Shift Parallelism 可以通过与 SwiftKV 和 Speculative Decoding 等 SoTA 推理技术组合，加速实际生产部署，提供最快的开源推理解决方案（完成时间缩短 3.4x，吞吐量提高 1.06x），

5. DP、TP 和 Shift Parallelism 的成本细分分析，并探索密集模型和稀疏模型之间的进一步权衡。

4.1 实验设置

4.1.1 硬件。除非另有说明，我们使用每个具有 8xH200 GPU 的 AWS 实例，即 p5en.48xlarge。每个 GPU 具有 141 GB 内存和 4.8 TB/s 带宽，还提供

1,979 FP8 TFLOPS 与张量核心的峰值密集矩阵乘法。GPU 与额定带宽为 900 GB/s 的 NVSwitch 网络互连。

4.1.2 软件。除非另有说明，我们使用插入 vLLM v0.9.2 的实现（第 3.4 节）。为了进行比较，我们使用 SGLang [24] v0.4.6、TRT-LLM [13] v0.18.2。

表 4. 评估中使用的模型。

Model Name	Num. Params.	Num. Lay.	Hidden Size	# Heads Q	# Heads KV
Llama-70B	70B	80	8192	64	8
Qwen-32B	32B	64	5120	64	8
Llama-17B-16E	109B/17B	48	5120	40	8
Qwen-30B-A3B	30B/3B	48	2048	32	4

4.1.3 模型。我们使用表 4 中列出的模型，全部采用 FP8 量化。Shift Parallelism 最初是为密集模型设计的，因此我们首先对 L70B 和 Q32B 进行主要评估，然后讨论专家混合 (MoE) 模型 Q30B-A3B 和 L17B-16E 的性能限制，它们的静态和活动参数数量分别显示在表 4 中。

4.1.4 数据集。我们按演示顺序使用以下数据集。i) 类似于现实生活生产环境的突发合成模式，ii) 一些开源生产痕迹（详细信息在第 4.2.2 节中给出），

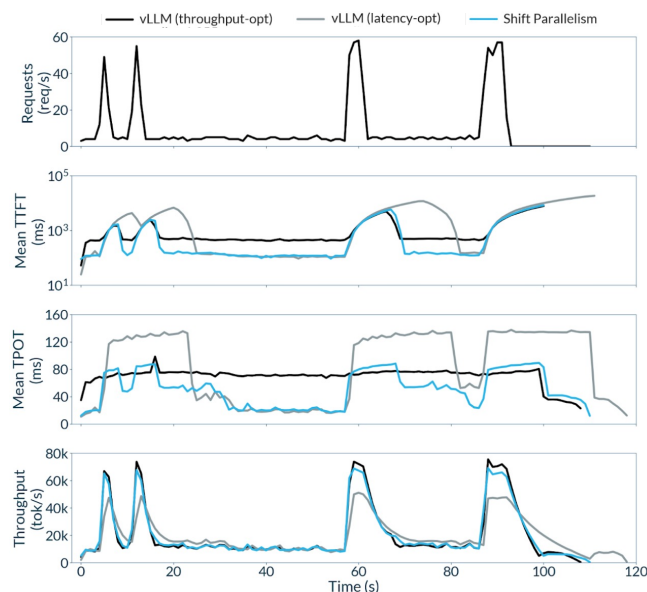


图 7. 移位并行在动态流量下实现了最低的响应、最快的生成和接近最佳的吞吐量。我们使用了 Llama-70B 和修改版本的 vLLM 服务基准，使请求成为低频的稳定请求流，偶尔突发高频请求。

iii) synthetic requests with random data for parameterized benchmarking, and iv) a trace that is a mixture of requests from HumanEval [3] and from a CodeAct agent [11] running against SWEbench [8] (HumanEval is one-shot and SWEbench is agentic). We filter these requests to create a synthetic dataset requests for the sake of running speculative decoding in Section 4.5.

4.2 Latency and Throughput in Real-World Traffic

4.2.1 Bursty Synthetic Workload. For testing Shift Parallelism on real-life environment, we create a bursty dataset by changing the arrival using vLLM’s burstiness benchmark. Figure 7 (top) shows resulting traffic pattern that has four high-traffic bursts. The rest of the results show the input latency (i.e., TTFT) and the output latency (i.e., TPOT) that is experienced by a request in milliseconds, and also the combined input/output token throughput of all requests in tokens per second.

To obtain Figure 7, we randomly mix two real-life datasets that are described in Section 4.1.4. The mix involves both latency- and throughput-critical requests with variable sizes.

Table 5 summarizes the latency and throughput statistics that are collected from the trace of the bursty workload experiment (Figure 7). The experiment trace with vLLM’s TP and DP, and also the proposed Shift Parallelism shows

- Shift Parallelism obtains the lowest latency across TP and DP with bursty (dynamic) traffic since it can sustain low latency with a higher traffic than the other two .
- Shift Parallelism obtains a higher peak throughput than TP, and therefore processes the batches in a shorter time. As a result, the wait time for latency-critical request is reduced significantly, i.e., TTFT does not explode with Shift Parallelism (148 ms vs. 3.9 sec.).

Table 5. Performance stats with the bursty workload.

	Median TTFT	Median TPOT	Peak Throughput
vLLM (throughput opt.—DP)	1,355 ms	83 ms	75,535 tok/s
vLLM (latency opt.—TP)	3,930 ms	85 ms	51,162 tok/s
vLLM+Shift Parallelism	148 ms	51 ms	69,147 tok/s

Overall, Shift Parallelism can handle the high-traffic bursts better than both TP and DP, achieving up to $9.16\times$ lower TTFT and $1.63\times$ lower TPOT than both, while at the same time achieving nearly as good throughput as DP during high-traffic periods, ultimately, improving the quality of service.

4.2.2 Real-Life Workload Traces. In this section, we study two real-life traces. We performed these case studies on code completion and conversation on a single node based on the following open-source traces: 1) Azure LLM Code Trace [15], and 2) Mooncake Conversation Trace [17]. For

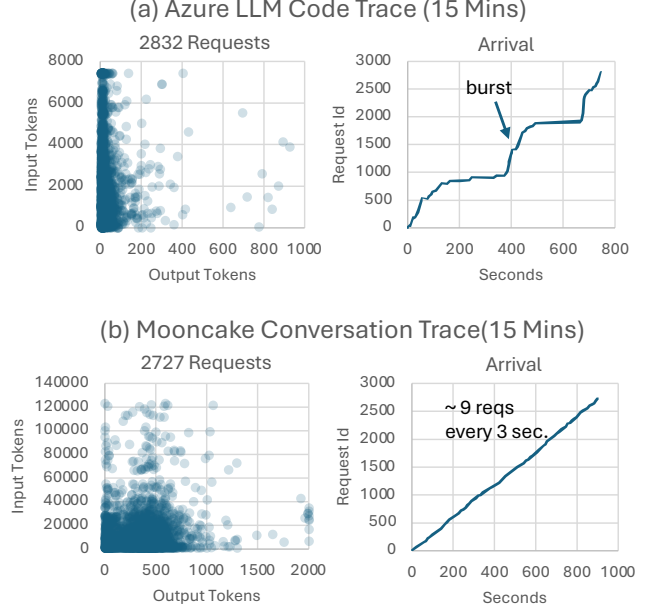


Figure 8. Input/output distribution and arrival times of real-world requests. (a) represents a bursty workload due to agentic code completion, causing low-traffic (silent) and high-traffic (burst) regions. (b) represents a steady arrival of medium input, long output, where a batch of nearly 9 requests is sent every 3 seconds.

demonstration we run both traces for 15 minutes. The resulting request distribution and arrival rates are shown in Figure 8.

Azure LLM Code Trace on Llama-70B. This trace is produced on Azure platform from real-world agentic code generation. We replay these traces on our platform (Section 4.1.1) with the Llama-70B-FP8 model.

As seen in Figure 9, DP (shown with green) is throughput oriented, and handles the bursts better compared to TP (shown with yellow), yielding a lower TTFT and completion time in high traffic. TP is latency oriented and yields a lower TPOT, and hence completion time than DP in low traffic.

Shift Parallelism (Figure 9, red) is efficient across traffic and obtains the lowest TTFT, TPOT, and hence completion time. The statistics in Figure 11 (a) show that shift parallelism helps to achieve tighter service-level objectives (e.g., p50, p99), compared to other parallelisms.

Mooncake Conversation Trace on Qwen-32B. This trace is from real-world conversation on Moonshot AI’s platform. In this case, we chose to use a smaller model (Qwen-32B-FP8) because the Llama model did not sustain the traffic and context size in our platform, i.e., the arrival rate is so high for a single node that KV cache becomes full, causing wait times as seen in DP and TP in Figure 10.

iii) 使用随机数据进行参数化基准测试的合成请求，以及 iv) 来自 HumanEval [3] 和针对 SWEBench [8] 运行的 CodeAct 代理 [11] 的请求混合的跟踪（HumanEval 是一次性的，SWEBench 是代理的）。我们过滤这些请求以创建合成数据集请求，以便在第 4.5 节中运行推测解码。

4.2 现实世界流量中的延迟和吞吐量

4.2.1 突发合成工作负载。为了在现实环境中测试移位并行性，我们通过使用 vLLM 的突发基准更改到达来创建突发数据集。图 7（顶部）显示了具有四个高流量突发的最终流量模式。其余结果显示请求所经历的输入延迟（即 TTFT）和输出延迟（即 TPOT）（以毫秒为单位），以及所有请求的组合输入/输出令牌吞吐量（以每秒令牌数为单位）。

为了获得图 7，我们随机混合第 4.1.4 节中描述的两个现实数据集。该组合涉及具有可变大小的延迟和吞吐量关键请求。

表 5 总结了从突发工作负载实验跟踪中收集的延迟和吞吐量统计数据（图 7）。vLLM 的 TP 和 DP 的实验轨迹以及所提出的 Shift Parallelism 表明

- 移位并行性在具有突发（动态）流量的 TP 和 DP 上获得最低延迟，因为它可以比其他两者更高的流量维持低延迟。
- Shift Parallelism 比 TP 获得更高的峰值吞吐量，因此可以在更短的时间内处理批次。因此，延迟关键请求的等待时间显着减少，即 TTFT 不会因移位并行性而爆炸（148 毫秒 vs. 3.9 秒）。

表 5. 突发工作负载的性能统计数据。

	Median TTFT	Median TPOT	Peak Throughput
vLLM (throughput opt.—DP)	1,355 ms	83 ms	75,535 tok/s
vLLM (latency opt.—TP)	3,930 ms	85 ms	51,162 tok/s
vLLM+Shift Parallelism	148 ms	51 ms	69,147 tok/s

总体而言，Shift Parallelism 可以比 TP 和 DP 更好地处理高流量突发，实现比两者低 9.16× 的 TTFT 和低 1.63× 的 TPOT，同时在高流量期间实现几乎与 DP 一样好的吞吐量，最终提高服务质量。

4.2.2 现实生活中的工作负载跟踪。在本节中，我们研究两个现实生活中的痕迹。我们基于以下开源跟踪对单个节点上的代码完成和对话进行了这些案例研究：1) Azure LLM 代码跟踪 [15] 和 2) Mooncake 对话跟踪 [17]。为了

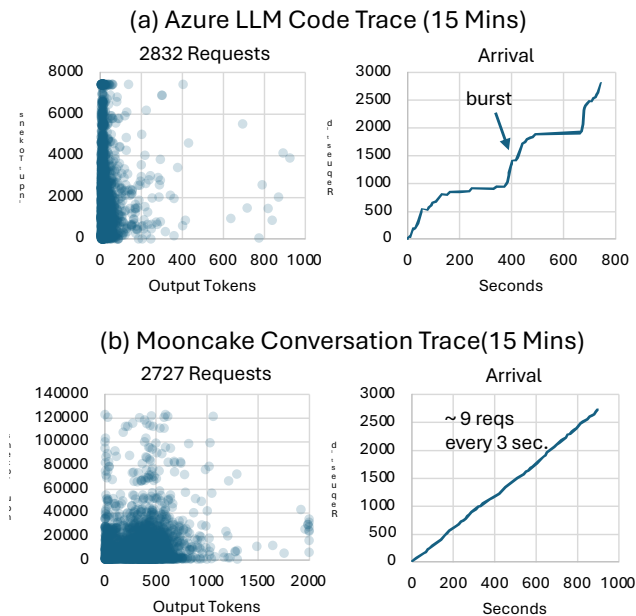


图 8. 实际请求的输入/输出分布和到达时间。(a) 表示由于代理代码完成而导致的突发工作负载，导致低流量（静默）和高流量（突发）区域。(b) 表示中输入、长输出的稳定到达，每 3 秒发送一批近 9 个请求。

演示中我们运行两条跟踪 15 分钟。由此产生的请求分布和到达率如图 8 所示。

Llama-70B 上的 Azure LLM 代码跟踪。该跟踪是在 Azure 平台上根据真实世界的代理代码生成而生成的。我们使用 Llama-70B-FP8 模型在我们的平台（第 4.1.1 节）上重播这些跟踪。

如图 9 所示，DP（以绿色显示）是面向吞吐量的，与 TP（以黄色显示）相比，可以更好地处理突发，从而在高流量时产生较低的 TTFT 和完成时间。TP 是面向延迟的，在低流量时产生的 TPOT 较低，因此完成时间比 DP 低。

移位并行性（图 9，红色）在整个流量中非常高效，并获得最低的 TTFT、TPOT，从而获得最低的完成时间。图 11 (a) 中的统计数据表明，与其他并行性相比，轮班并行性有助于实现更严格的服务级别目标（例如，p50、p99）。

Qwen-32B 上的月饼对话轨迹。该痕迹来自 Moonshot AI 平台上的真实对话。在这种情况下，我们选择使用较小的模型（Qwen-32B-FP8），因为 Llama 模型无法维持我们平台中的流量和上下文大小，即单个节点的到达率如此之高，以至于 KV 缓存已满，导致等待时间，如图 10 中的 DP 和 TP 所示。

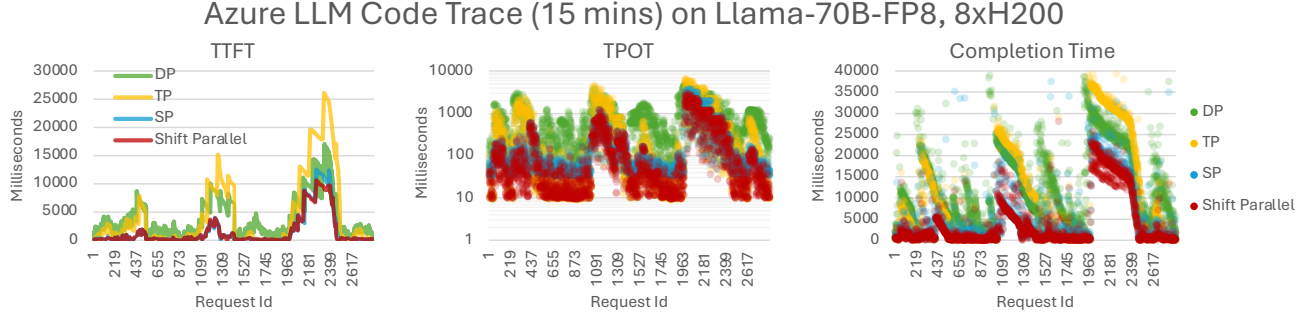


Figure 9. The Azure trace has three prominent bursts (see requests 437, 1091, 2181) and those coincide with the bursts experiencing a higher TTFT and TPOT, and hence the completion time increases in during the bursts. The requests at the beginning of a burst experience a higher completion time because their decode overlaps with the prefill of consecutive requests.

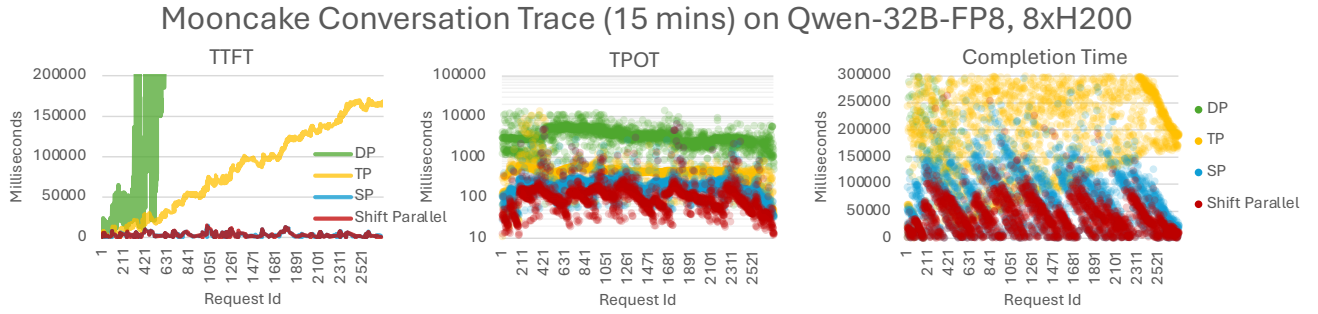


Figure 10. The Mooncake trace involves a heavier workload. DP and TP cannot keep up with the traffic, and cause wait times to grow indefinitely as seen in TTFT. SP and Shift Parallel can sustain the conversation traffic with a finite completion time.

In our initial try, Qwen-32B-FP8 did not sustain the conversation traffic. Therefore we turned on FP8 KV cache data type (originally it is FP16) for increasing the KV cache capacity. The wait times were reduced, yet DP and TP still could not sustain the incoming traffic yet Shift Parallelism could run the trace without KV cache overflow. Therefore only SP and Shift Parallel solutions can support this trace on a single node without creating significant queuing delays. The statistics are shown in Figure 11 (b).

4.3 Performance Benchmarks

In this section we send synthetic requests for presenting performance characteristics via parameterized experiments.

4.3.1 Latency vs. throughput tradeoff. We evaluate the latency vs. throughput tradeoff across parallelism using a uniform request size of 4k input tokens and 250 output tokens. For finding the peak throughput, we send a batch of requests (thousands) and provide sufficient concurrency to saturate the GPU throughput. For finding the lowest latency, we process requests sequentially, i.e., a single request at a time.

Figure 12 compares DP, TP, SP, and Shift Parallelism across Llama-70B and Qwen-32B. Shift Parallelism achieves the lowest TTFT that is 1.56 $\times$ and 6 $\times$ lower than TP and DP

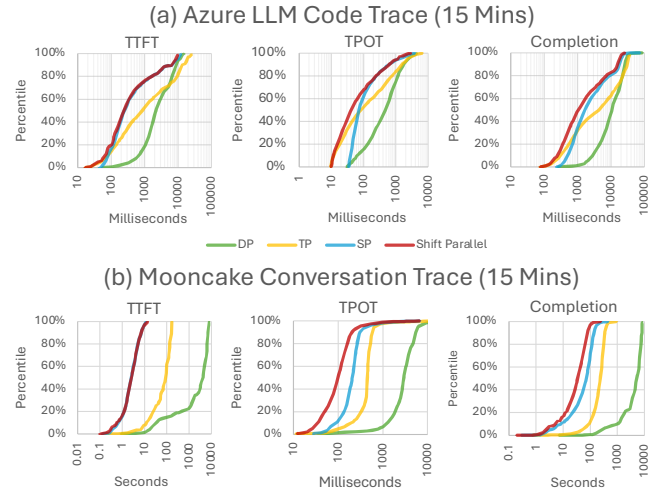


Figure 11. Request latency distributions in the (a) Azure (b) Mooncake traces. Shift Parallelism is more likely to deliver the lowest completion time in either case.

for Llama, and 4.45 $\times$ and 1.31 $\times$ for Qwen. Shift Parallelism achieves that lowest TPOT that is 9.34 ms for Llama and 8.68 ms for Qwen. Shift Parallelism experiences significantly less throughput degradation compared to TP. Specifically,

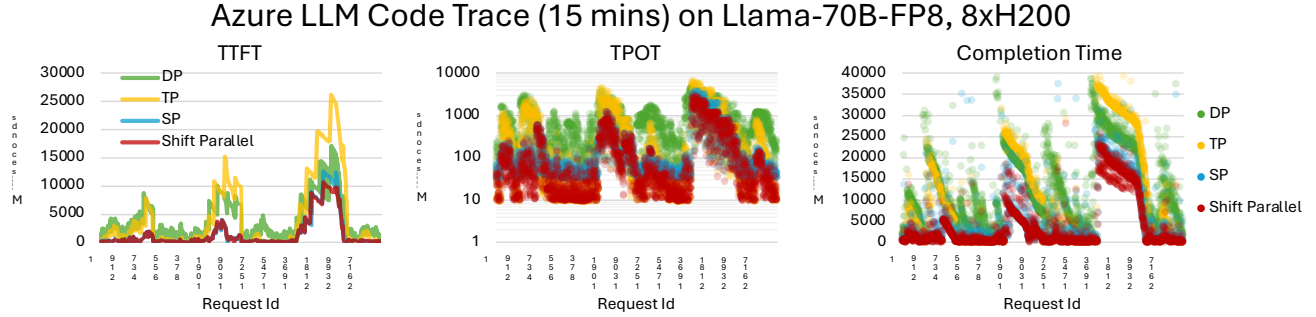


图 9. Azure 跟踪具有三个突出的突发（请参阅请求 437、1091、2181），这些突发与经历较高 TTFT 和 TPOT 的突发一致，因此突发期间的完成时间会增加。突发开始时的请求经历较长的完成时间，因为它们的解码与连续请求的预填充重叠。

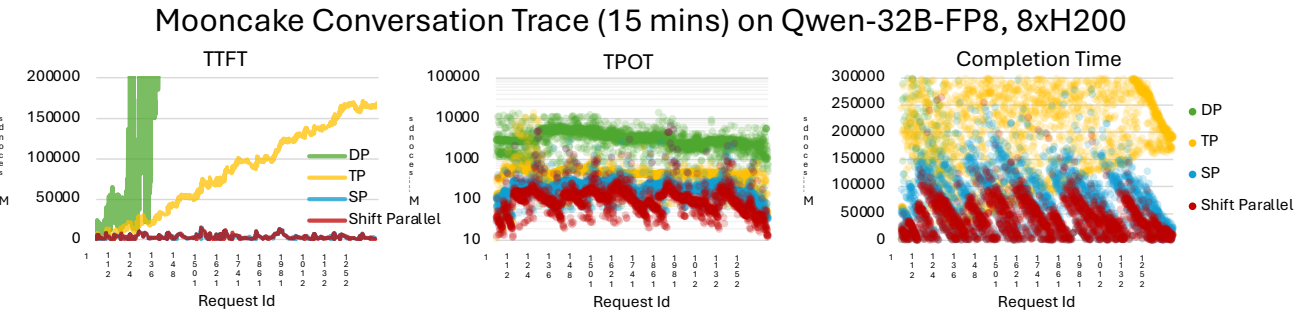


图 10. Mooncake 跟踪涉及较重的工作负载。DP 和 TP 无法跟上流量，导致等待时间无限增长，如 TTFT 中所示。SP 和 Shift Parallel 可以在有限的完成时间内维持会话流量。

在我们最初的尝试中，Qwen-32B-FP8 无法维持对话流量。因此我们开启 FP8 KV 缓存数据类型（原来是 FP16）来增加 KV 缓存容量。等待时间减少了，但 DP 和 TP 仍然无法维持传入流量，但移位并行可以在没有 KV 缓存溢出的情况下运行跟踪。因此，只有 SP 和 Shift Parallel 解决方案可以在单个节点上支持此跟踪，而不会产生明显的排队延迟。统计数据如图 11(b) 所示。

4.3 性能基准

在本节中，我们发送综合请求，通过参数化实验来呈现性能特征。

4.3.1 延迟与吞吐量的权衡。我们使用 4k 输入令牌和 250 个输出令牌的统一请求大小来评估跨并行性的延迟与吞吐量权衡。为了找到峰值吞吐量，我们发送一批请求（数千个）并提供足够的并发性以使 GPU 吞吐量饱和。为了找到最低的延迟，我们按顺序处理请求，即一次处理一个请求。

图 12 比较了 Llama-70B 和 Qwen-32B 的 DP、TP、SP 和移位并行性。Shift Parallelism 实现最低 TTFT，比 TP 和 DP 低 1.56 $\times$ 和 6 $\times$

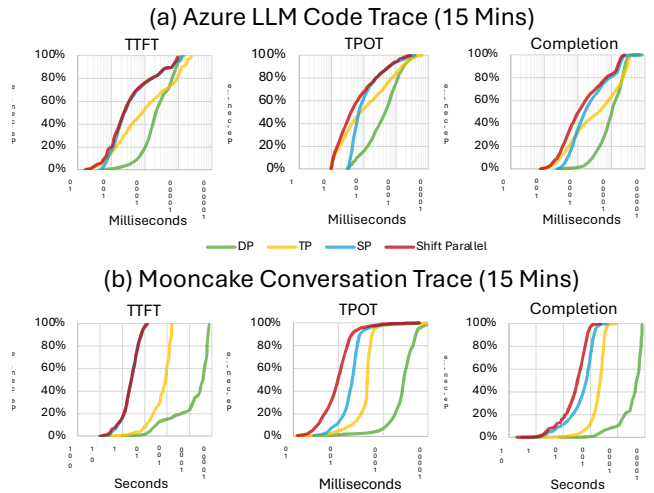


图 11. (a) Azure (b) Mooncake 跟踪中的请求延迟分布。在这两种情况下，移位并行性更有可能提供最短的完成时间。

对于 Llama，以及对于 Qwen 4.45 $\times$ 和 1.31 $\times$ 。移位并行性实现了最低 TPOT，Llama 为 9.34 毫秒，Qwen 为 8.68 毫秒。与 TP 相比，Shift Parallelism 的吞吐量下降显著着减少。具体来说，

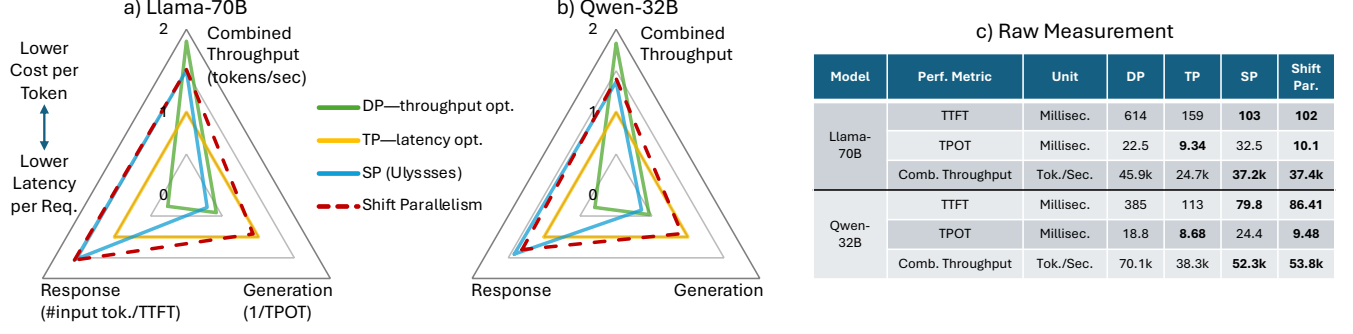


Figure 12. Comparison of response and generation latency, and throughput, all in tokens/sec. with (a) Llama-70B and (b) Qwen-32B based on the (c) measurements. Shift Parallelism simultaneously obtains a higher throughput and a lower latency than TP, alleviating the latency vs. throughput tradeoff of existing parallelisms.

TP loses 46% and 45% throughput with Llama and Qwen, yet Shift Parallelism only loses 18% and 23%, respectively. A bit part of it comes from vLLM’s engine overhead that we discuss in Section 4.4.

4.3.2 Variations across context sizes. For investigating TTFT, TPOT, and throughput with various input context sizes, we repeat the experiment in Figure 12 for input sequences with 2k–128k tokens and 250 output tokens. Figure 17 presents the input and output latency in low traffic, and throughput in high traffic.

Shift Parallelism provides a 6.97× and 1.56× faster response than DP and TP, respectively, because it uses SP for prefill, which is more efficient than both DP and TP. As a result, Shift Parallelism is more responsive and also provides

a faster completion time especially for long input and short output contexts where TTFT dominates the completion time (such as in summarization).

Shift Parallelism provides up to 2.45× faster generation than DP and a similar output latency to TP. Ideally, the output latency should not depend on the input context size in theory, but in practice TPOT increases with the input size (see Figure 13). The main reason is that each output token needs to read more number of tokens from the KV cache as input context grows, and eventually the system becomes memory bandwidth bound. TP and Shift Parallelism parallelize the attention layer (Section 2.3), and hence the KV cache, providing memory bandwidth, mitigating the output latency for long inputs.

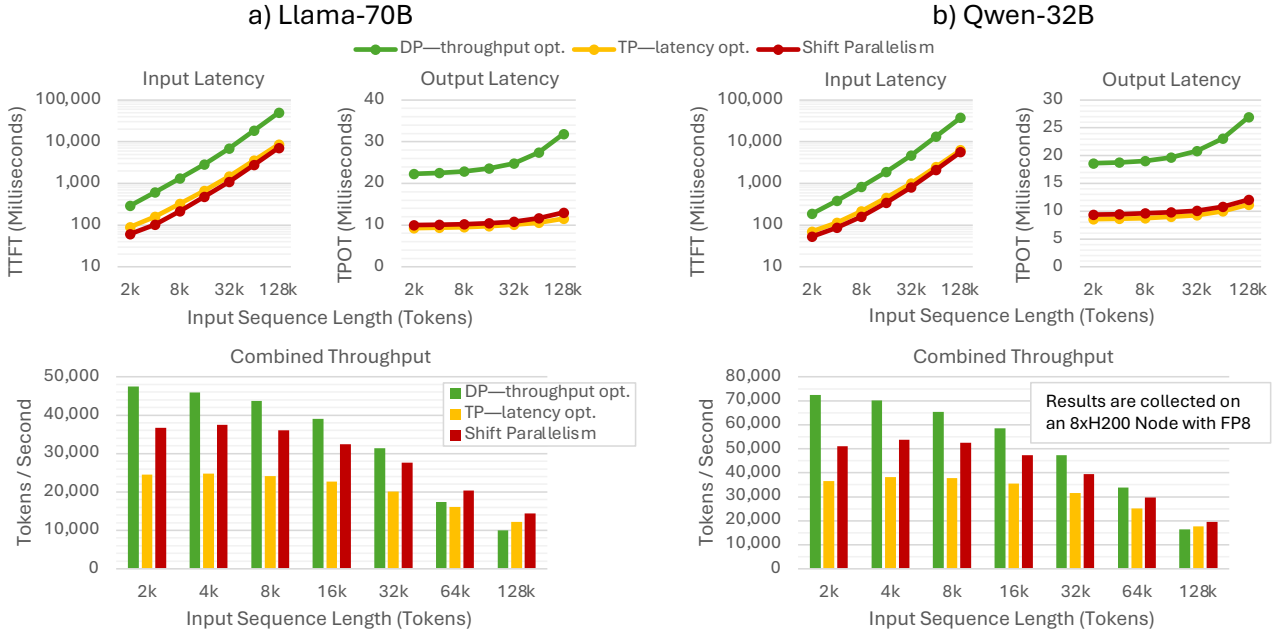


Figure 13. Performance variation across input sequence length: Minimum latency (TTFT, TPOT), and maximum throughput of (a) Llama-70B and (b) Qwen-32B. The throughput drops with large context sizes due to the excess attention time.

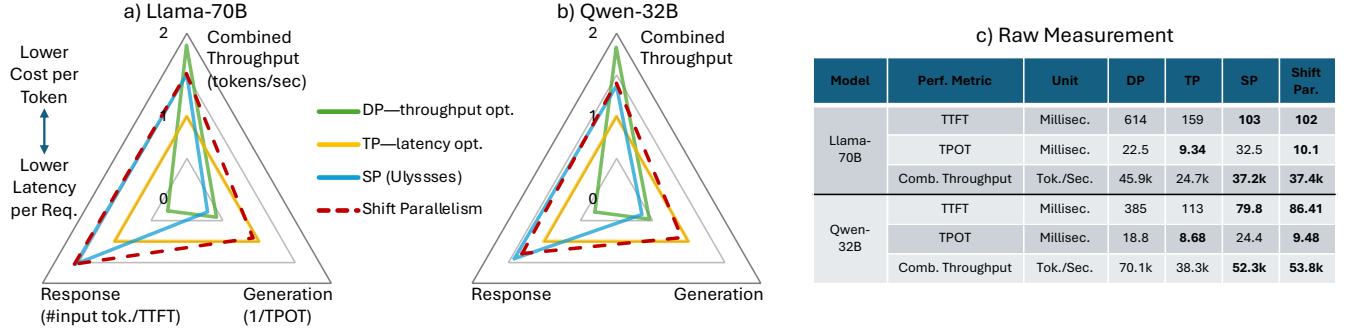


图 12. 响应和生成延迟以及吞吐量的比较（均以令牌/秒为单位）。(a) Llama-70B 和 (b) Qwen-32B 基于 (c) 测量。与 TP 相比，移位并行同时获得了更高的吞吐量和更低的延迟，从而减轻了现有并行性的延迟与吞吐量的权衡。

Llama 和 Qwen 的 TP 吞吐量损失了 46% 和 45%，而 Shift Parallelism 仅分别损失了 18% 和 23%。其中一部分来自 vLLM 的引擎开销，我们将在 4.4 节中讨论。

4.3.2 上下文大小的变化。为了研究不同输入上下文大小的 TTFT、TPOT 和吞吐量，我们对具有 2k–128k 令牌和 250 个输出令牌的输入序列重复图 12 中的实验。图 17 显示了低流量时的输入和输出延迟以及高流量时的吞吐量。

Shift Parallelism 的响应速度分别比 DP 和 TP 快 6.97× 和 1.56×，因为它使用 SP 进行预填充，这比 DP 和 TP 都更高效。因此，Shift Parallelism 响应速度更快，并且还提

更快的完成时间，特别是对于长输入和短输出上下文，其中 TTFT 主导完成时间（例如在摘要中）。

移位并行性的生成速度比 DP 快 2.45×，并且输出延迟与 TP 类似。理想情况下，理论上输出延迟不应取决于输入上下文大小，但实际上 TPOT 随着输入大小而增加（见图 13）。主要原因是，随着输入上下文的增长，每个输出令牌需要从 KV 缓存中读取更多数量的令牌，最终系统会受到内存带宽的限制。TP 和 Shift Parallelism 并行化注意力层（第 2.3 节），从而并行化 KV 缓存，提供内存带宽，减轻长输入的输出延迟。

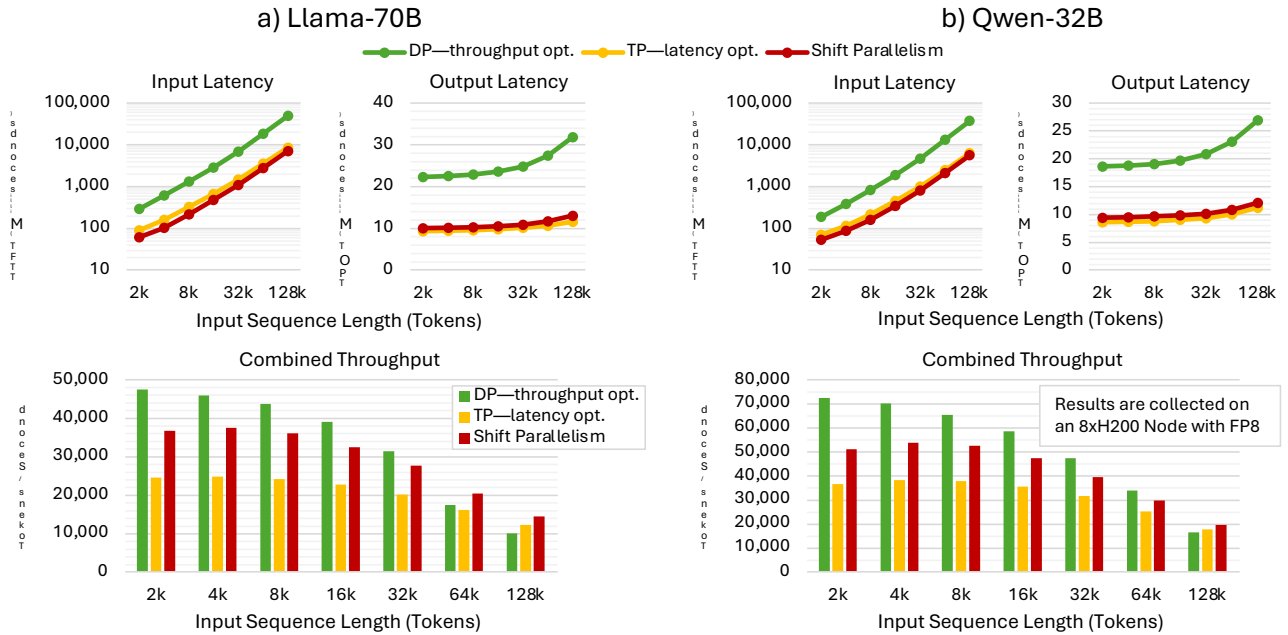


图 13. 输入序列长度的性能变化：(a) Llama-70B 和 (b) Qwen-32B 的最小延迟（TTFT、TPOT）和最大吞吐量。由于注意力时间过多，吞吐量随着上下文大小的增加而下降。

Shift Parallelism obtains up to $1.51\times$ higher peak throughput than TP, meaning that processing the high-traffic bursts and also batch workloads is approx. 50% faster with Shift Parallelism. Nevertheless, the throughput drops significantly with larger contexts because attention time dominates the end-to-end generation. See Section 4.4 for analysis.

4.3.3 Latency vs. Arrival Rate. To investigate the performance between extremely high and low traffic rates, we test Shift Parallelism across a wide range of intermediate traffic by varying the request arrival rates. We measure TTFT and TPOT of an individual request, which both increases with higher traffic, and calculate the completion time as $\text{TTFT} + \# \text{output tok.} \times \text{TPOT}$. The question is, where does the tradeoff happen, and how well Shift Parallelism transitions from latency to throughput optimization?

Results in Figure 14 shows the performance variation across arrival rates. TP and DP curves cross over at a critical arrival rate (a few req/sec). Yet Shift Parallelism guarantees the lowest latency regardless of the arrival rate—strictly better than both DP and TP solutions. In low-to-medium rates (req/s), Shift Parallelism switches back-and-forth across SP and TP for minimizing the input (TTFT) and output (TPOT) latencies, respectively. In high traffic, Shift Parallelism uses SP to save combined throughput (tokens/sec).

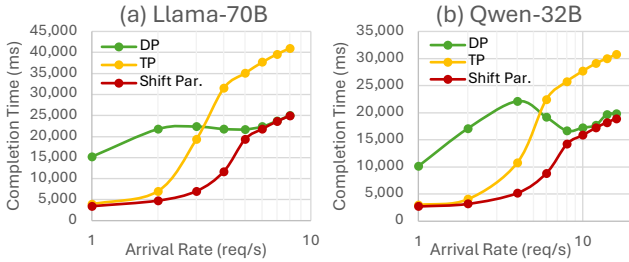


Figure 14. Request completion time vs. arrival rate. TP and DP make the performance tradeoff across arrival rates. Shift Parallelism strictly obtains the lowest completion time across arrival rates. Request size: 8k input, 250 output.

4.4 Cost Breakdown

We analyze the cost of individual system components by taking away one component at a time. Figure 15 shows the resulting breakdown of time to process a batch of requests with Llama-70B and Qwen-32B models on a single node. We clearly see that SP (and hence Shift Parallelism) has a lower communication cost than TP. On the other hand, we observe two unaddressed performance bottlenecks that are not related to Shift Parallelism:

- Attention time grows significantly with the sequence size, and therefore reduces the combined throughput. Recent papers address this issue using sparse attention [4] and it is out of scope of this paper.

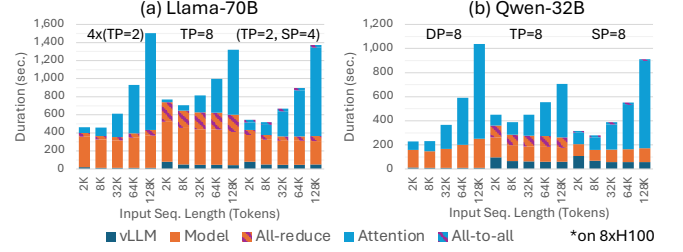


Figure 15. End-to-end cost breakdown* of time spent in a batch workload with (a) Llama-70B and (b) Qwen-32B. Shorter seq. $\rightarrow$ vLLM overhead, longer seq. $\rightarrow$ attn. time.

- The parallelization cost of vLLM is significant in small models (e.g., compare Llama-70B, Qwen-32B). We find vLLM cost by removing the forward pass. This indicates that a large portion of the remaining throughput gap between DP and SP might actually be the vLLM overhead unrelated to SP.

4.5 Shift Parallelism in Production

We fully integrated Shift Parallelism in our existing production environment (see Section 3.4). Running efficiently in production is not only about parallelism, but it also requires a plethora of other state-of-the-art techniques. To that extent, we integrated Shift Parallelism with SwiftKV [16] and speculative decoding [14, 21] in our production environment [19].

Figure 16 shows that our combined production simultaneously achieves highest throughput (lowest cost) and lowest completion time—all in one deployment—outperforming the best open source systems optimized for each metric individually². The compounding effect of SwiftKV and speculative decoding are denoted.

²We enabled the best available speculative decoding for each framework. These experiments were run on data sets generated using real-world production traces to compute throughput, and a mixture of ShareGPT, HumanEval and SWEbench to measure latency. As a result, these results are representative of performance achievable in real-world deployments.

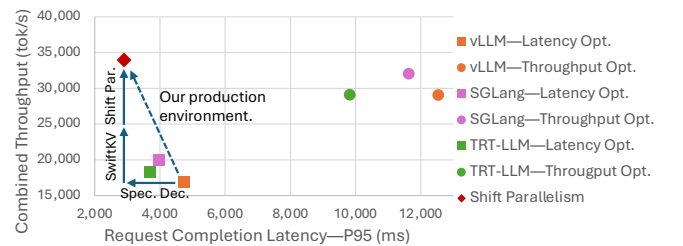


Figure 16. Compounding effect of optimizations and comparison of our production environment with other frameworks out-of-the-box using a real dataset on Llama-70B. Latency- and throughput-optimized configurations use TP and DP, respectively.

移位并行比 TP 获得高出 1.51 $\times$ 的峰值吞吐量，这意味着处理高流量突发和批处理工作负载的时间约为 1.51 $\times$ 。使用 Shift Parallelism 速度提高 50%。然而，随着上下文的增大，吞吐量会显著下降，因为注意力时间在端到端生成中占主导地位。分析见 4.4 节。

4.3.3 延迟与到达率。为了研究极高和极低流量之间的性能，我们通过改变请求到达率来测试各种中间流量的移位并行性。我们测量单个请求的 TTFT 和 TPOT，它们都随着流量的增加而增加，并将完成时间计算为 TTFT + #output tok. $\times$ TPOT。问题是，权衡发生在哪里，以及移位并行性从延迟到吞吐量优化的过渡效果如何？

图 14 中的结果显示了不同到达率之间的性能变化。TP 和 DP 曲线以临界到达率（几个请求/秒）交叉。然而，无论到达率如何，移位并行性都能保证最低的延迟——明显优于 DP 和 TP 解决方案。在中低速率 (req/s) 下，移位并行性在 SP 和 TP 之间来回切换，以分别最小化输入 (TTFT) 和输出 (TPOT) 延迟。在高流量中，移位并行使用 SP 来节省组合吞吐量（令牌/秒）。

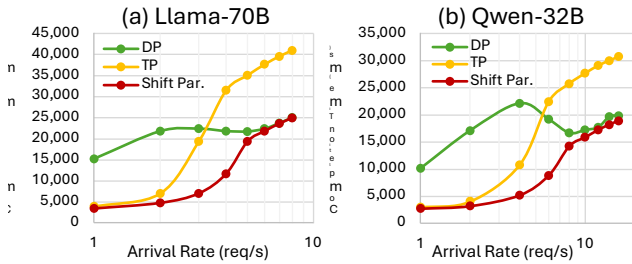


图 14. 请求完成时间与到达率。TP 和 DP 在到达率之间进行性能权衡。轮班并行性严格获得跨到达率的最低完成时间。请求大小：8k 输入，250 输出。

4.4 成本明细

我们通过一次去掉一个组件来分析单个系统组件的成本。图 15 显示了在单个节点上使用 Llama-70B 和 Qwen-32B 模型处理一批请求的时间细分。我们清楚地看到 SP（以及移位并行性）的通信成本比 TP 更低。另一方面，我们观察到两个与移位并行性无关的未解决的性能瓶颈：

- 注意力时间随着序列大小的增加而显著增加，因此降低了组合吞吐量。最近的论文使用稀疏注意力 [4] 解决了这个问题，但这超出了本文的范围。

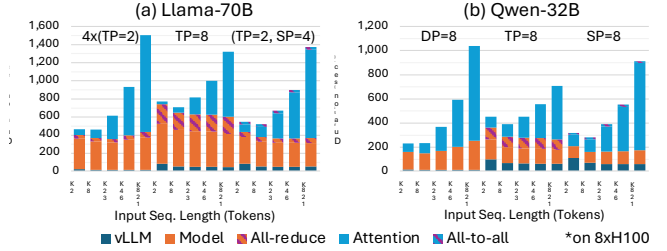


图 15. (a) Llama-70B 和 (b) Qwen-32B 批处理工作负载所花费时间的端到端成本细分*。较短的序列。→ vLLM 开销，更长的序列。→ 附件。时间。

- vLLM 的并行化成本在小型模型中非常显著（例如，比较 Llama-70B、Qwen-32B）。我们通过删除前向传播来找到 vLLM 成本。这表明 DP 和 SP 之间剩余吞吐量差距的很大一部分实际上可能是与 SP 无关的 vLLM 开销。

4.5 生产中的轮班并行性

我们将 Shift Parallelism 完全集成到现有的生产环境中（参见第 3.4 节）。在生产中高效运行不仅涉及并行性，还需要大量其他最先进的技术。在这个意义上，我们在生产环境 [19] 中将移位并行性与 SwiftKV [16] 和推测解码 [14, 21] 集成。

图 16 显示，我们的组合生产同时实现了最高吞吐量（最低成本）和最短完成时间（全部在一次部署中），其性能优于针对每个指标单独优化的最佳开源系统²。指出了 SwiftKV 和推测解码的复合效应。

²We enabled the best available speculative decoding for each framework. These experiments were run on data sets generated using real-world production traces to compute throughput, and a mixture of ShareGPT, HumanEval and SWEbench to measure latency. As a result, these results are representative of performance achievable in real-world deployments.

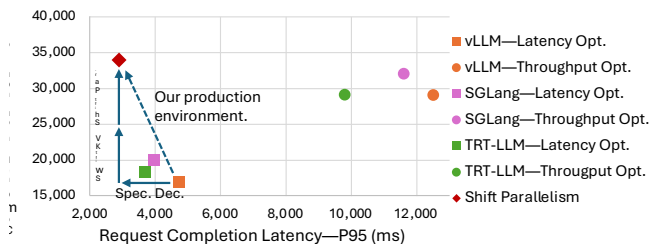


图 16. 使用 Llama-70B 上的真实数据集，优化的复合效果以及我们的生产环境与其他开箱即用的框架的比较。延迟优化和吞吐量优化的配置分别使用 TP 和 DP。

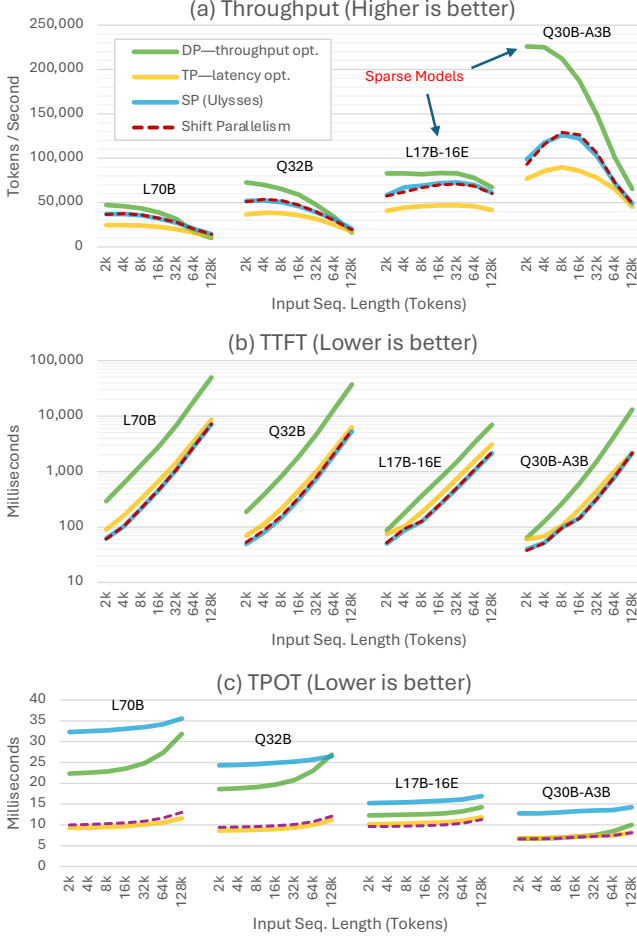


Figure 17. (a) Peak Throughput and minimum latency—(b) TTFT and (c) TPOT—comparison across parallelisms, models and input sequence lengths. Shift Parallelism switches across SP and TP for obtaining high throughput and low latency.

4.6 Limitations and Future Work

To investigate further on Shift Parallelism’s performance behavior, we stretch our evaluation to two recently released sparse (i.e., MoE) models that are listed in the bottom two rows of Table 4.

To enable these models, we use the SP generalizations in Section 3: i) Llama-17B-16E barely fits into a single GPU and when $SP = 8$ is used in the base config., there is no memory left in the KV cache to support large context sizes. To enable long contexts, we use Algorithm 1 for the base config. ($SP = 4, TP = 2$). ii) Qwen-30B-A3B suffers from scaling because it only has 4 KV heads, and we use KV cache replication (Section 3.2) to scale the model across $SP = 8$ GPUs.

Figure 17 compares the throughput and latency performance in across the board. The models sorted from larger to smaller. The sparse models attain a higher throughput and lower latency than dense models, simply because they

have fewer active parameters (see Table 4). Compared to TP, Shift parallelism shows excellent performance on both sparse models: Up to 50% higher throughput without increasing the latency.

The smallest model, Qwen-30B-A3B, attains a much higher throughput (225k tokens/sec.) than the other models with DP, and the throughput suddenly drops either with TP and SP, especially with smaller end of input sizes. The discrepancy is due to the vLLM’s parallelization overhead with small models that is discussed in Section 4.4.

The sparse models beg further investigation in the context of expert parallelism (EP). Specifically, there is no prior work that combines SP with EP to further optimize sparse models, which we will leave as a future work.

5 Related Work

The heterogeneous resource demands of LLM inference is one of its main challenges, and prior works have proposed different techniques to manage it. We discuss a few works related to Shift Parallelism in this section.

Chunked Prefill [1, 6] tackles the heterogeneous resource demand of LLM inference requests in their input-processing (prefill) stage vs their output-generation (decode) phase. During prefill, the input tokens are processed in parallel and is thus compute-intensive, while during decode, a single token is generated at a time but requires the full KV-cache of prior tokens, and is thus memory-intensive. Chunked prefill is proposed by DeepSpeed-FastGen [6] and Sarathi-Serve [1] and mixes requests in both prefill and decode phases in the same batch, thereby improving resource utilization and combined throughput. Today, it is default in many popular inference engines [9, 13, 24]. In comparison, Shift Parallelism is targeted at time-varying sizes of each batch. It is orthogonal and compatible with chunked prefill, and our experiments in Sec. 4 are run using their combination.

Disaggregated Inference [18, 25] uses separate GPU workers for prefill and decode so that prefill throughput and decode latency can be separately scaled and optimized. Compared with chunked-prefill systems and Shift Parallelism, disaggregated inference can eliminate interference between requests in the prefill and decode stages, but at the cost of dedicating additional resources to each stage. Additionally, the KV cache must be communicated from a prefill worker to a decode worker for each request, causing extra communication overhead. In contrast, Shift Parallelism with chunked-prefill overlaps prefill and decode, with decode tokens accessing the KV cache from local memory, resulting in more efficient resource utilization and less cost per token.

6 Conclusion

LLM’s have diverse real-world applications that yield different traffic patterns that have different performance requirements. Fundamentally, existing parallelisms (TP and

DP) for inference optimize either for latency (for interactive applications) or throughput (for batch workloads), but do not optimize for multiple applications in the same deployment.

For supporting dynamic workloads, we need to switch back-and-forth between parallelisms, yet cannot switch across TP and DP due to their KV cache mismatch. As a remedy, we bring in SP (Ulysses) that is originally applied to training for high throughput, and generalized it for inference, and providing the KV cache invariance across SP and TP by addressing corner cases. We call our solution Shift Parallelism, where a deployment has two configurations. that switch back and forth across SP (base config.) and TP (shift config.).

Our extensive benchmarking shows that Shift Parallelism addresses the latency vs. throughput tradeoff in a low cost way, providing up to 50% more throughput compared without losing latency across traffic rates. We fully integrated Shift Parallelism along with other SoTA that we use in production, and open source our inference system.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. arXiv:2403.02310 [cs.LG] <https://arxiv.org/abs/2403.02310>
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Felipe Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 4895–4901. doi:10.18653/v1/2023.emnlp-main.298
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgén Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [4] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. *ArXiv abs/1904.10509* (2019). <https://arxiv.org/abs/1904.10509>
- [5] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs.CL] <https://arxiv.org/abs/2312.10997>
- [6] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. arXiv:2401.08671 [cs.PF] <https://arxiv.org/abs/2401.08671>
- [7] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. arXiv:2309.14509 [cs.LG] <https://arxiv.org/abs/2309.14509>
- [8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 [cs.CL] <https://arxiv.org/abs/2310.06770>
- [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180 [cs.LG] <https://arxiv.org/abs/2309.06180>
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401 [cs.CL] <https://arxiv.org/abs/2005.11401>
- [11] Graham Neubig and Xingyao Wang. 2024. OpenHands CodeAct 2.1: An Open, State-of-the-Art Software Development Agent. *All Hands AI Blog* (1 November 2024). <https://www.all-hands.dev/blog/openhands-codeact-21-an-open-state-of-the-art-software-development-agent>
- [12] Junichiro Niimi. 2024. Dynamic Sentiment Analysis with Local Large Language Models using Majority Voting: A Study on Factors Affecting Restaurant Evaluation. arXiv:2407.13069 [cs.CL] <https://arxiv.org/abs/2407.13069>
- [13] NVIDIA Developer. 2023. NVIDIA TensorRT-LLM: An Open-Source Library for Accelerating LLM Inference. <https://developer.nvidia.com/blog/optimizing-inference-on-llms-with-tensorrt-llm-now-publicly-available/>. Accessed: August 19, 2025.
- [14] Gabriele Oliaro, Zhihao Jia, Daniel Campos, and Aurick Qiao. 2025. SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications. arXiv:2411.04975 [cs.CL] <https://arxiv.org/abs/2411.04975>
- [15] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st International Symposium on Computer Architecture (ISCA '24)*. ACM. doi:10.1145/3649329.3655655 Dataset available at https://github.com/Azure/AzurePublicDataset/blob/master/data/AzureLLMInferenceTrace_code.csv.
- [16] Aurick Qiao, Zhewei Yao, Samyam Rajbhandari, and Yuxiong He. 2025. SwiftKV: Fast Prefill-Optimized Inference with Knowledge-Preserving Model Transformation. arXiv:2410.03960 [cs.LG] <https://arxiv.org/abs/2410.03960>
- [17] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, USA. <https://github.com/kvccache-ai/Mooncake> Dataset file: https://github.com/kvccache-ai/Mooncake/blob/main/FAST25-release/traces/conversation_trace.jsonl.
- [18] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. arXiv:2407.00079 [cs.DC] <https://arxiv.org/abs/2407.00079>
- [19] Samyam Rajbhandari, Mert Hidayetoglu, Aurick Qiao, Ye Wang, Juncheng Yang, Jeff Rasley, Michael Wyatt, and Yuxiong He. 2025. Arctic Inference with Shift Parallelism: Fast and Efficient Open Source Inference System for Enterprise AI. arXiv:2507.11830 [cs.DC] <https://arxiv.org/abs/2507.11830>

DP) 用于推理优化延迟 (对于交互式应用程序) 或吞吐量 (对于批量工作负载), 但不会针对同一部署中的多个应用程序进行优化。

为了支持动态工作负载, 我们需要在并行性之间来回切换, 但由于 KV 缓存不匹配, 无法跨 TP 和 DP 切换。作为补救措施, 我们引入了最初应用于高吞吐量训练的 SP (Ulysses), 并将其推广到推理, 并通过解决极端情况提供跨 SP 和 TP 的 KV 缓存不变性。我们将我们的解决方案称为 “Shift Parallelism”, 其中部署有两种配置。在 SP (基本配置) 和 TP (转移配置) 之间来回切换。

我们广泛的基准测试表明, 移位并行性以低成本方式解决了延迟与吞吐量的权衡问题, 与不同流量速率的延迟相比, 可提高高达 50% 的吞吐量。我们将 Shift Parallelism 与我们在生产中使用的其他 SoTA 完全集成, 并开源了我们的推理系统。

参考

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov 和 Ramachandran Ramjee. 2024. 使用 Sarathi-Serve 控制 LLM 推理中的吞吐量与延迟权衡。arXiv:2403.02310 [cs.LG] <https://arxiv.org/abs/2403.02310> [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Felipe Lebrón 和 Sumit Sanghai. 2023. GQA: 从多头检查点训练通用多查询变压器模型。2023 年自然语言处理经验方法会议论文集。计算语言学协会, 新加坡, 4895-4901. doi:10.18653/v1/2023.emnlp-main.298 [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, 吉里什·萨斯特里、帕梅拉·米什金、布鲁克·陈、斯科特·格雷、尼克·莱德、米哈伊尔·巴甫洛夫、Aletha Power、卢卡斯·凯泽、穆罕默德·巴伐利亚、克莱门斯·温特、菲利普·蒂莱、费利佩·佩特罗斯基·萨奇、戴夫·卡明斯、马蒂亚斯·普拉珀特、福蒂奥斯·尚齐斯、伊丽莎白·巴恩斯、阿里尔·赫伯特·沃斯、威廉·赫伯根·格斯、亚历克斯·尼科尔、Alex Paino、Nikolas Tezak、Jie Tang、Ig or Babuschkin、Suchir Balaji、Shantanu Jain、William Saunders、Christopher Hesse、Andrew N. Carr、Jan Leike、Josh Achiam、Vedant Misra、Evan Morikawa、Alec Radford、Matthew Knight、Miles Brundage、Mira Murati、Katie Mayer、Peter Welinder、Bob McGrew、Dario Amodei、Sam McCandlish、Ilya Sutskever 和 Wojciech Zaremba。2021。评估在代码上训练的大型语言模型。arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374> [4] Rewon Child, Scott Gray, Alec Radford 和 Ilya Sutskever。2019。用稀疏变压器生成序列。ArXiv abs/1904.10509 (2019)。 <https://arxiv.org/abs/1904.10509> [5] 高云帆, 熊云, 高新宇, 贾康祥, 潘金柳, 毕玉玺, 戴毅, 孙家伟, 王猛, 王浩芬。2024. 大型语言模型的检索增强生成: 调查。arXiv:2312.10997 [cs.CL] <https://arxiv.org/abs/2312.10997> [6] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyangqin, Arash Bakhtiari, Lev Krilenko 和 Yuxiong He。2024. DeepSpeed-FastGen: 通过 MII 和 DeepSpeed-Inference 为大语言模型提供高通量文本。arXiv:2401.08671 [cs.PF] <https://arxiv.org/abs/2401.08671>
- [7] Sam Ade Jacobs, Masahiro Tanaka, 张成明, 张敏佳, Shuaiwen Leon Song, Samyam Rajbhandari 和 Yuxiong He。2023. DeepSpeed Ulysses: 用于实现极长序列变压器模型训练的系统优化。arXiv:2309.14509 [cs.LG] <https://arxiv.org/abs/2309.14509> [8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press 和 Karthik Narasimhan。2024. SWE-bench: 语言模型可以解决现实世界的 GitHub 问题吗? arXiv:2310.06770 [cs.CL] <https://arxiv.org/abs/2310.06770> [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Shen, Lianmin Cheng, Codyhao Yu, Joseph E. Gonzalez, Hao Zhu 和 Ion Stoica。2023. 使用 PagedAttention 进行大型语言模型的高效内存管理。arXiv:2309.06180 [cs.LG] <https://arxiv.org/abs/2309.06180> [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel 和 Douwe Kiela。2021. 知识密集型 NLP 任务的检索增强生成。arXiv:2005.11401 [cs.CL] <https://arxiv.org/abs/2005.11401> [11] Graham Neubig 和 Xingyao Wang。2024 年。OpenHands Code Act 2.1: 开放、最先进的软件开发代理。All Hands AI 博客 (2024 年 11 月 1 日)。 <https://www.all-hands.dev/blog/openhands-codeact-21-an-open-state-of-the-art-software-development-agent> [12] Junichiro Niimi。2024. 使用多数投票的本地大语言模型的动态情感分析: 影响餐厅评价因素的研究。arXiv:2407.13069 [cs.CL] <https://arxiv.org/abs/2407.13069> [13] NVIDIA 开发人员。2023. NVIDIA TensorRT-LLM: 用于加速 LLM 推理的开源库。 <https://developer.nvidia.com/blog/optimizing-inference-on-llms-with-tensorrt-llm-now-publicly-available/>。访问时间: 2025 年 8 月 19 日。 [14] Gabriele Oliaro, Zhihao Jia, Daniel Campos 和 Aurick Qiao。2025. SuffixDecoding: 新兴人工智能应用的极端推测解码。arXiv:2411.04975 [cs.CL] <https://arxiv.org/abs/2411.04975> [15] Pratyush Patel, Esha Choukse, Chaojie 张, Aashaka Shah, Íñigo Goiri, Saeed Maleki 和 Ricardo Bianchini。2024. Splitwise: 使用相分裂的高效生成法学硕士推理。第 51 届国际计算机体系结构研讨会 (ISCA '24) 论文集。ACM. doi:10.1145/3649329.3655655 数据集位于 https://github.com/Azure/AzurePublicDataset/blob/master/data/AzureLLMInferenceTrace_code.csv。 [16] Aurick Qiao, Zhewei Yao, Samyam Rajbhandari, 何宇雄。2025. SwiftKV: 具有知识保存模型转换的快速预填充优化推理。arXiv:2410.03960 [cs.LG] <https://arxiv.org/abs/2410.03960> [17] 秦若宇, 李哲明, 何蔚然, 崔家磊, 任峰, 张明星, 吴永伟, 郑伟民和徐欣然。2025. Mooncake: 用更多的存储换取更少的计算——一种以 KVCache 为中心的架构, 用于服务 LLM 聊天机器人。第 23 届 USENIX 文件和存储技术会议 (FAST 25) 的会议记录。USENIX 协会, 美国加利福尼亚州圣克拉拉。 https://github.com/kvcache-ai/Mooncake/blob/main/FAST25-release/traces/conversation_trace.jsonl。 [18] 秦若愚, 李哲明, 何蔚然, 张明星, 吴永伟, 郑伟民, 徐欣然。2024. Mooncake: 用于 LLM 服务的以 KVCache 为中心的分解架构。arXiv:2407.00079 [cs.DC] <https://arxiv.org/abs/2407.00079> [19] Samyam Rajbhandari, Mert Hidayetoglu, Aurick Qiao, Ye Wang, Jun Cheng Yang, Jeff Rasley, Michael Wyatt 和 Yuxiong He。2025. 带有移位并行性的北极推理: 用于企业人工智能的快速高效的开源推理系统。arXiv:2507.11830 [cs.DC] <https://arxiv.org/abs/2507.11830>

- [20] Snowflake AI Research. 2025. ArcticInference: A vLLM plugin for low-latency, high-throughput LLM inference. <https://github.com/snowflakedb/ArcticInference>.
- [21] Ye Wang, Gabriele Oliaro, Jaeseong Lee, Yuxiong He, Aurick Qiao, and Rajbhandari Samyam. 2025. Fastest Speculative Decoding in vLLM with Arctic Inference and Arctic Training. <https://www.snowflake.com/en/engineering-blog/fast-speculative-decoding-vllm-arctic>.
- [22] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. arXiv:2401.07339 [cs.SE] <https://arxiv.org/abs/2401.07339>
- [23] Wenxuan Zhang, Yue Deng, Bing Liu, Sinno Jialin Pan, and Lidong Bing. 2023. Sentiment Analysis in the Era of Large Language Models: A Reality Check. arXiv:2305.15005 [cs.CL] <https://arxiv.org/abs/2305.15005>
- [24] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] <https://arxiv.org/abs/2312.07104>
- [25] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. arXiv:2401.09670 [cs.DC] <https://arxiv.org/abs/2401.09670>

A Artifact Appendix

A.1 Abstract

Our evaluation in Section 4 involves extensive results. For assessing the real-world benefit, we will focus on running open-source production traces (Section 4.2.2). The artifact compares vLLM’s built-in parallelisms (DP and TP) and the proposed parallelisms (SP and Shift Parallelism) using ArcticInference plug-in system that we also use in our production. For reproducibility, we included the instructions in the repository, and uploaded the cleaned traces into a permanent archive.

A.2 Artifact check-list (meta-information)

- **Algorithm:** Multi-GPU parallelism strategies (DP, TP, SP, Shift Parallel) for LLM inference.
- **Program:** vLLM v0.10.1 and the corresponding ArcticInference plug-in version (both are open source).
- **Model:** We use two open-source models downloadable from Hugging Face.
- **Data set:** Two open-source production traces. Clean traces for reproducibility are archived in DOI: <https://doi.org/10.5281/zenodo.18240909>.
- **Run-time environment:** Linux, Python 3.10
- **Hardware:** A node with 8×H200 Nvidia GPUs w/ NVLinks.
- **Metrics:** TTFT, TPOT, completion time
- **Output:** vLLM’s benchmarking script prints the result.
- **Experiments:** Running vLLM benchmarking scripts (server and client). We provide the scripts.
- **How much disk space required (approximately)?:** 100 GB to store both models.
- **How much time is needed to prepare workflow (approximately)?:** 15 mins

- **How much time is needed to complete experiments (approximately)?:** 5 node-hours. Experiments can be easily done in parallel across two nodes.
- **Publicly available?:** <https://github.com/snowflakedb/ArcticInference>
- **Code licenses?:** Apache-2

A.3 Description

A.3.1 How to access. vLLM is open source (<https://github.com/vllm-project/vllm>).

A.3.2 Hardware dependencies. We used the hardware described in Section 4.1.1), yet a generic Nvidia DGX-H200 node is sufficient for 1:1 reproduction. Optimal configurations, and hence the results may look different another type of multi-GPU node, yet the conclusion should be the same.

A.3.3 Software dependencies. We depend on vLLM v0.10.1 and the corresponding ArcticInference plug-in (commit 5e08f0f).

A.3.4 Data sets. We use the following datasets (links given):

- Azure LLM code
- Mooncake conversation

We provide the cleaned traces (15 Mins each) at DOI: <https://doi.org/10.5281/zenodo.18240909> that reproduces Figures 9–10. We also include a vLLM benchmarking patch for running these traces in our repository.

A.3.5 Models. The models are open-source in Hugging Face (links given):

1. RedHatAI/Llama-3.3-70B-Instruct-FP8-dynamic
2. Qwen/Qwen3-32B-FP8

These can be downloaded as `huggingface-cli download <model> -local-dir <model>`

A.4 Installation

We built our work on vLLM, therefore it is essential to make vLLM work first. The installation details can be found in <https://github.com/vllm-project/vllm>, yet it should be as easy as: `pip install vllm==v0.10.1`

A.5 Experiment workflow

The step-by-step instructions can be found in ArcticInference/benchmark/reproducibility. First the output file of an experiment should be obtained. Then the plotting script should be used to plot TTFT, TPOT, and completion time. The scripts for running the experiments and plotting them as in Figure 9–10 are provided.

A.6 Evaluation and expected results

Shift parallelism is expected to have a lower TTFT, TPOT, and completion compared to TP and DP. The raw files can be plotted using `plot.py`. Additionally, the columns (TTFT, TPOT, Completion) can be sorted to obtain Figure 11.

[20] 雪花人工智能研究。2025. ArcticInference: 用于低延迟、高吞吐量 LLM 推理的 vLLM 插件。 <https://github.com/snowflakedb/ArcticInference>. [21] 王野, Gabriele Oliaro, Jaeseong Lee, 何宇雄, Aurick Qiao, Rajbhandari Samyam. 2025. vLLM 中最快的推理解码与北极推理和北极训练。 <https://www.snowflake.com/en/engineering-blog/fast-speculative-decoding-vllm-arctic>. [22] 张克池, 李佳, 李革, 史贤杰, 金志. 2024.CodeAgent: 通过工具集成代理系统增强代码生成, 应对现实世界的回购级编码挑战。 arXiv:2401.07339 [cs.SE] <https://arxiv.org/abs/2401.07339> [23] 张文轩, 邓岳, 刘兵, 潘辛诺嘉林和冰立东. 2023. 大语言模型时代的情感分析: 现实检验。 arXiv:2305.15005 [cs.CL] <https://arxiv.org/abs/2305.15005> [24] Lianmin Cheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Codyhao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Yingheng. 2024.SGLang: 结构化语言模型程序的高效执行。 arXiv:2312.07104 [cs.AI] <https://arxiv.org/abs/2312.07104>

[25] 钟银民, 刘胜宇, 陈俊达, 胡建波, 朱一波, 刘玄哲, 金鑫, 张浩. 2024.DistServe: 分解预填充和解码以实现 Goodput 优化的大型语言模型服务。 arXiv:2401.09670 [cs.DC] <https://arxiv.org/abs/2401.09670>

神器附录

A.1 摘要

我们在第 4 节中的评估涉及广泛的结果。为了评估现实世界的效益, 我们将重点关注运行开源生产跟踪 (第 4.2.2 节)。该工件使用我们也在生产中使用的 ArcticInference 插件系统, 比较了 vLLM 的内置并行性 (DP 和 TP) 和建议的并行性 (SP 和 Shift 并行性)。为了可重复性, 我们将说明包含在存储库中, 并将清理后的痕迹上传到永久存档中。

A.2 工件检查表 (元信息)

- 算法: 用于 LLM 推理的多 GPU 并行策略 (DP、TP、SP、Shift Parallel)。
- 程序: vLLM v0.10.1 和相应的 ArcticInference 插件版本 (均为开源)。
- 模型: 我们使用两个开源模型, 可从 Hugging Face 下载。
- 数据集: 两个开源生产轨迹。清晰的可重复性痕迹存档在 DOI 中: <https://doi.org/10.5281/zenodo.18240909>。
- 运行环境: Linux、Python 3.10
- 硬件: 具有 8xH200 Nvidia GPU 和 NVLink 的节点。
- 指标: TTFT、TPOT、完成时间
- 输出: vLLM 的基准测试脚本打印结果。
- 实验: 运行 vLLM 基准测试脚本 (服务器和客户端)。我们提供脚本。
- 需要多少磁盘空间 (大约)? : 100 GB 来存储两个模型。
- 准备工作流程需要多长时间 (大约)? : 15 分钟

- 完成实验需要多少时间 (大约)? : 5 个节点小时。实验可以轻松地跨两个节点并行完成。
- 公开可用吗? : <https://github.com/snowflakedb/ArcticInference>
- 代码许可证? : Apache-2

A.3 描述

A.3.1 如何访问。vLLM 是开源的 (<https://github.com/vllm-project/vllm>)。

A.3.2 硬件依赖性。我们使用第 4.1.1 节中描述的硬件, 但通用 Nvidia DGX-H200 节点足以进行 1:1 复制。最佳配置, 因此结果可能看起来与另一种类型的多 GPU 节点不同, 但结论应该是相同的。

A.3.3 软件依赖性。我们依赖于 vLLM v0.10.1 和相应的 ArcticInference 插件 (提交 5e08f0f)。

A.3.4 数据集。我们使用以下数据集 (给出的链接):

- Azure 大语言模型
- 月饼对话

我们在 DOI 上提供清理后的痕迹 (每条 15 分钟): <https://doi.org/10.5281/zenodo.18240909>, 再现图 9-10。我们还包含一个 vLLM 基准测试补丁, 用于在我们的存储库中运行这些跟踪。

A.3.5 模型。这些模型在 Hugging Face 中是开源的 (已给出链接): 1. RedHatAI/Llama-3.3-70B-Instruct-FP8-dynamic 2. Qwen/Qwen3-32B-FP8 这些可以作为 Huggingface-cli download <model> --local-dir <model> 下载

A.4 安装

我们的工作建立在 vLLM 的基础上, 因此首先让 vLLM 发挥作用至关重要。安装详细信息可以在 <https://github.com/vllm-project/vllm> 中找到, 但它应该像以下一样简单: `pip install vllm==v0.10.1`

A.5 实验流程

分步说明可以在 ArcticInference/benchmark/reproducibility 中找到。首先应获取实验的输出文件。然后应该使用绘图脚本来绘制 TTFT、TPOT 和完成时间。提供了用于运行实验并绘制图的脚本, 如图 9-10 所示。

A.6 评价和预期结果

与 TP 和 DP 相比, 移位并行性预计具有较低的 TTFT、TPOT 和完成度。可以使用 plot.py 绘制原始文件。此外, 还可以对列 (TTFT、TPOT、Completion) 进行排序以获得图 11。